

Книга: MicroPython за VK_RA4M2 чрез демонстрационни примери

Цел

Тази книга е направена върху реално наличния пакет от 67 примерни .py файла за VK_RA4M2.

Тук целта не е само да се изброят файловете.

Целта е да се покрият всички глави, теми и примери от плана, без пропуски и без измислени ресурси.

Затова книгата е организирана в три слоя:

1. Ресурсна карта на борда и порта.
2. Матрица за пълност спрямо учебния план.
3. Подробни глави с кодови откъси и препратки към всички реални сорс файлове.

Как да се ползва книгата

Книгата се чете заедно със сорс файловете в examples.

Препоръчителният ритъм е:

1. Прочитате теорията в съответната глава.
2. Стартирате показания минипример.
3. Отваряте пълния .py файл.
4. Преглеждате ресурсния header в началото му.
5. Променяте само по една стойност и наблюдавате ефекта.

Когато примерът съдържа предпазен флаг като DO_WRITE, DO_BOOTLOADER, DO_LIGHTSLEEP, DO_DEEPSLEEP или SET_DEMO_TIME, започвайте с безопасната стойност.

Съдържание

Тази секция дава бърз преглед на всички части и глави в книгата.

Част I: Първи стъпки с борда и с MicroPython

- Глава 1. Първа програма
- Глава 2. Първи асинхронни примери

Част II: Езикови основи на MicroPython

- Глава 3. Променливи и константи
- Глава 4. Оператори, условия и цикли

- [Глава 5. Enums в MicroPython](#)
- [Глава 6. Масиви и таблици](#)
- [Глава 7. Структури в MicroPython](#)
- [Глава 8. Указатели и еквивалентите им в MicroPython](#)
- [Глава 9. Функции, callbacks и таблици от функции](#)

Част III: Цифрови и аналогови входове и изходи

- [Глава 10. Цифров изход](#)
- [Глава 11. Цифров вход](#)
- [Глава 12. Аналогов вход и ADC](#)
- [Глава 13. PWM](#)

Част IV: Таймери, прекъсвания и време

- [Глава 14. Закъснения, хардуерни таймери и Timer\(-1\)](#)
- [Глава 15. Прекъсвания](#)
- [Глава 16. Дебаунс на бутон](#)
- [Глава 17. Четене на бутон: polling, IRQ, Timer\(-1\), debounce и събития](#)
- [Глава 18. RTC](#)

Част V: Сериен интерфейс

- [Глава 19. I2C master и runtime избор на пинове](#)
- [Глава 20. SoftI2C и I2CTarget](#)
- [Глава 21. Hardware SPI и SoftSPI](#)

Част VI: TouchPad, памет и системни функции

- [Глава 22. TouchPad: базово четене](#)
- [Глава 23. TouchPad: диагностика, cached API и асинхронно следене](#)
- [Глава 24. renesas.Flash, /flash и dataflash](#)
- [Глава 25. Управление на паметта в MicroPython](#)
- [Глава 26. machine модулът и порт-специфично поведение](#)

Част VII: Индикация и входни матрици

- [Глава 27. Динамична 7-сегментна индикация и матрично сканиране](#)

Част VIII: FSM като начин на мислене

- [Глава 28. Минимална FSM и таблична FSM](#)
- [Глава 29. FSM с timeout](#)
- [Глава 30. Conditions vs Events](#)
- [Глава 31. Guard функции](#)
- [Глава 32. Приоритети и event queue](#)

- Глава 33. Tickless timers и sleep идея

Част IX: Реален пример

- Глава 34. Управление на осветление и вентилация за баня
- Глава 35. Декомпозиция на две FSM

Част X: UART комуникация

- Глава 36. Базова UART комуникация

Част XI: Външни модули по интерфейси

- Глава 37. Работа с външни модули по интерфейси

Част XII: asyncio като учебна секция

- Глава 38. Основи на asyncio
- Глава 39. asyncio Event синхронизация

Част XIII: Специализирани светлинни модули

- Глава 40. NeoPixel
- Глава 41. WS2812 през machine.WS2812 и SCI TX-only

Ресурсна карта на VK_RA4M2

Книгата и примерите приемат следната карта на ресурси:

- LED: 1 брой -> LED1 = P204
- Бутон: 1 брой -> SW1 = P400
- USB device пинове: USBDP = P914, USBDM = P915, USB_VBUS = P407
- ADC външни входове: 13 броя -> P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500
- ADC вътрешни източници: 3 броя -> ADC.CORE_TEMP, ADC.CORE_VREF, ADC.VREF
- DAC изходи: 2 броя -> P014, P015
- PWM изходи: 14 броя -> P107, P106, P105, P104, P113, P114, P112, P115, P608, P409, P408, P600, P304, P303
- UART: 4 инстанции -> UART(0), UART(2), UART(7), UART(9)
- I2C master: 2 инстанции -> I2C(0)=P400/P401, I2C(1)=P100/P101
- I2CTarget: 2 инстанции -> I2CTarget(0)=P400/P401, I2CTarget(1)=P100/P101
- SPI: 1 канал -> CS=P103, SCK=P102, MISO=P100, MOSI=P101
- SoftI2C: наличен
- SoftSPI: наличен
- TouchPad входове: 12 броя -> P205, P206, P407, P408, P409, P410, P411, P412, P413, P414, P415, P708

- CTSU специален пин: P207 = TSCAP
- Хардуерни таймери: 2 броя -> Timer(1), Timer(2)
- Софтуерен таймер: Timer(-1)
- RTC: 1 брой
- Вътрешна файлова система: /flash
- renesas.Flash: наличен block device
- dataflash: 8 KB

Важни уточнения:

- SPI тук се разглежда като 1 канал.
- Timer(-1) тук се разглежда като софтуерен таймер.
- SW1 е върху P400, което влиза в конфликт с I2C(0).SCL.
- LED1 е active-low: логическа 0 го светва, логическа 1 го гаси.
- Pin.PULL_UP се ползва в курса; Pin.PULL_DOWN не се разглежда като наличен ресурс в тези уроци.
- lightsleep и deepsleep са показани предпазливо.

Матрица за пълност спрямо плана

Това е контролният списък, по който книгата е проверена.

- Примерна програма -> examples/20_language_basics/01_sample_program.py
- Променливи и константи -> examples/20_language_basics/02_variables_and_constants.py
- Аритметични оператори -> examples/20_language_basics/03_operators_conditions_loops.py
- Логически оператори -> examples/20_language_basics/03_operators_conditions_loops.py
- if / if-else / if-elif-else -> examples/20_language_basics/03_operators_conditions_loops.py
- goto в MicroPython липсва -> обяснено в главата за управление на изпълнението
- switch-case в MicroPython липсва -> обяснено в главата за управление на изпълнението
- while -> examples/20_language_basics/03_operators_conditions_loops.py
- for -> examples/20_language_basics/03_operators_conditions_loops.py
- do-while еквивалент -> examples/20_language_basics/03_operators_conditions_loops.py
- Цифров изход -> examples/21_digital_io/01_digital_output.py, examples/01_blink_async.py
- Цифров вход -> examples/21_digital_io/02_digital_input.py
- Аналогов вход -> examples/22_analog/01_analog_input_adc.py
- Какво е ADC и как работи -> обяснено в главата за ADC и показано в examples/22_analog/01_analog_input_adc.py

- Измерване на ADC.read() с time.ticks_us() -> examples/23_timing/07_adc_timing_measure.py
- PWM изход -> examples/22_analog/02_pwm_output.py, examples/03_pwm_ab_same_channel.py
- Enums -> examples/20_language_basics/04_enums.py
- Масив -> examples/20_language_basics/05_arrays.py
- Масив от масиви -> examples/20_language_basics/05_arrays.py
- Структури -> examples/20_language_basics/06_structures.py
- Структури от структури -> examples/20_language_basics/06_structures.py
- Масив от структури -> examples/20_language_basics/06_structures.py
- Указатели като идея -> examples/20_language_basics/07_pointers_equivalents.py
- Указател към променлива/масив/структура -> еквиваленти чрез references и mutable обекти в examples/20_language_basics/07_pointers_equivalents.py
- Указател към указател -> обяснено като верига от references и индирекция чрез mutable обекти
- Масив от указатели -> еквивалент чрез списък от функции в examples/20_language_basics/08_functions.py
- Управление на паметта в MicroPython -> examples/28_machine_misc/04_memory_management.py, examples/20_language_basics/07_pointers_equivalents.py, examples/24_i2c/04_i2ctarget_memory.py
- gc.collect(), gc.mem_free(), gc.mem_alloc(), micropython.mem_info() -> examples/28_machine_misc/04_memory_management.py
- memoryview без копиране и преизползване на буфер -> examples/20_language_basics/07_pointers_equivalents.py, examples/28_machine_misc/04_memory_management.py, examples/24_i2c/04_i2ctarget_memory.py
- Функция -> examples/20_language_basics/08_functions.py
- Входни и изходни параметри -> examples/20_language_basics/08_functions.py
- Параметри чрез указател -> еквивалент чрез mutable обект в examples/20_language_basics/08_functions.py
- Указател към функция -> callback reference в examples/20_language_basics/08_functions.py
- Масив от указатели към функции -> operation_table в examples/20_language_basics/08_functions.py
- Таймери и закъснения -> examples/23_timing/01_delays_and_timers.py
- Софтуерен таймер -> examples/23_timing/04_software_timer_minus1.py
- Хардуерен срещу софтуерен таймер -> examples/23_timing/05_timer_compare_hw_sw.py
- time.ticks_us() и относителни измервания -> examples/23_timing/07_adc_timing_measure.py
- Прекъсвания -> examples/23_timing/02_interrupts.py

- Измерване на interrupt latency -> `examples/23_timing/08_interrupt_latency_measure.py`
- Приоритети -> `examples/30_fsm/06_priorities.py`
- Забраняване/разрешаване на прекъсвания -> `examples/23_timing/02_interrupts.py`,
`examples/28_machine_misc/02_machine_mem_and_bootloader.py`
- Четене на бутон и дебаунс -> `examples/23_timing/03_button_debounce.py`
- Четене на бутон с polling -> `examples/23_timing/09_button_polling_events.py`
- Четене на бутон по прекъсване с отложена обработка -> `examples/23_timing/10_button_irq_deferred.py`
- Четене на бутон със софтуерен Timer(-1) -> `examples/23_timing/11_button_soft_timer_hold.py`
- Събитие при натискане и отпускане -> `examples/23_timing/09_button_polling_events.py`,
`examples/23_timing/10_button_irq_deferred.py`
- Събитие при задържане -> `examples/23_timing/09_button_polling_events.py`,
`examples/23_timing/11_button_soft_timer_hold.py`
- HOLD_START и HOLD_REPEAT -> `examples/23_timing/11_button_soft_timer_hold.py`
- Дебаунс с битово изместване -> `examples/23_timing/12_button_shift_debounce.py`
- Динамична 7-сегментна индикация -> `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`
- Прекъсвания по таймер / времево опресняване -> обяснено в главата за display scan и timer главите
- Матрично сканиране на бутони -> `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`
- FSM идея -> `examples/30_fsm/01_fsm_lamp_button.py`
- Графично и таблично описание на FSM -> обяснено в главата за FSM и показано в `examples/30_fsm/02_fsm_table.py`
- FSM с timeout -> `examples/30_fsm/03_fsm_timeout.py`
- Conditions vs Events -> `examples/30_fsm/04_conditions_vs_events.py`
- Guard функции -> `examples/30_fsm/05_guard_functions.py`
- Реален пример: баня -> `examples/31_case_study/01_bathroom_control.py`
- Декомпозиция на 2 FSM -> `examples/31_case_study/02_two_fsms.py`
- Комуникация между стейт машини -> `examples/31_case_study/02_two_fsms.py`
- Event queue -> `examples/30_fsm/07_event_queue.py`
- Tickless timers + sleep -> `examples/30_fsm/08_tickless_sleep.py`,
`examples/28_machine_misc/03_sleep_modes.py`
- Работа с външни модули по интерфейси -> обяснено в главата за външни модули и показано в `examples/34_external_modules/01_i2c_module_probe.py`,
`examples/34_external_modules/02_spi_module_transaction.py`,
`examples/34_external_modules/03_uart_module_request_response.py`

- I2C scan и безопасен probe на модул -> `examples/34_external_modules/01_i2c_module_probe.py`
- SPI транзакция с CS рамка -> `examples/34_external_modules/02_spi_module_transaction.py`
- UART request/response с timeout -> `examples/34_external_modules/03_uart_module_request_response.py`
- NeoPixel индикация -> `examples/neopixel.py`, `examples/neopixel_test.py`
- Захранване на малък външен RGB модул от GPIO и еднопроводен data канал -> обяснено в главата за NeoPixel и показано в `examples/neopixel.py`
- WS2812 през специализиран C драйвер -> `examples/ws2812_sci.py`, `examples/ws2812_sci_test.py`, `examples/ws2812_sci_p101.py`, `examples/ws2812_sci_p109.py`
- WS2812 върху валиден SCI TX/MOSI pin без външен clock pin към модула -> обяснено в главата за machine.WS2812

Част I: Първи стъпки с борда и с MicroPython

Глава 1. Първа програма

Първата програма трябва да даде веднага видим резултат и да покаже как изглежда MicroPython файл за VK_RA4M2.

Ключова идея:

```
from machine import Pin # Импортираме класа Pin, чрез който управляваме GPIO пиновете.
import time # Импортираме модула time, за да правим закъснения между действията.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изходен обект за LED1 и започваме с изгасен LED.

for blink_index in range(3): # Обхождаме три последователни мигания с for цикъл.
    led.value(0) # Светваме LED1 като подадем активното за платката ниво.
    time.sleep_ms(300) # Изчакваме 300 милисекунди, за да се вижда светването.
    led.value(1) # Гасим LED1, за да завършим едно мигане.
    time.sleep_ms(300) # Изчакваме още 300 милисекунди преди следващото мигане.
```

Тук се виждат наведнъж:

- Pin
- for
- sleep_ms
- active-low поведението на LED1

Файлове в главата:

- `examples/20_language_basics/01_sample_program.py`

Резюме

- Главата показва най-малкия работещ MicroPython пример за платката.
- Фокусът е върху `Pin`, `sleep_ms()` и `active-low` поведението на LED1.
- Това е отправната точка за всички следващи практически уроци.

Самопроверка

1. Защо LED1 светва при `value(0)`?
2. Каква е ролята на `Pin.OUT` и `sleep_ms()` в примера?
3. Коя е най-малката промяна, с която бихте проверили, че разбирате кода?

Упражнения

1. Променете броя мигания и дължината на паузата и наблюдавайте резултата.
2. Направете различни времена за светнато и изгасено състояние.
3. Изнесете броя мигания и закъснението в именувани константи.

Ако не работи

- Симптом: LED1 не мига. Проверка: използвайте `Pin("LED1", Pin.OUT)` и имайте предвид `active-low` логиката.
- Симптом: ефектът е невидим. Проверка: вероятно паузите са твърде кратки или цикълът не се изпълнява както очаквате.

Нататък

След това мини към следващата глава, „Първи асинхронни примери“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 2. Първи асинхронни примери

Преди да влезем в детайлите на езика, полезно е да се види как изглежда неблокиращото мислене.

Минипример:

```
import asyncio # Импортираме asyncio, за да правим неблокиращи задачи в MicroPython.
from machine import Pin # Импортираме класа Pin, за да управляваме LED1 като цифров изход.

led = Pin("LED1", Pin.OUT, value=1) # Създаваме изход за LED1 и започваме от изгасено състояние.

async def blink_task(): # Дефинираме асинхронна задача, която ще мига без да блокира event loop-a.
    while True: # Повтаряме мигането безкрайно.
        led.value(0) # Светваме LED1.
        await asyncio.sleep_ms(500) # Отстъпваме управлението за 500 ms.
```

```
led.value(1) # Гасим LED1.  
await asyncio.sleep_ms(500) # Отново отстъпваме управлението за 500 ms.
```

Втори минипример:

```
async def count_seconds_task(): # Създаваме втора задача, която брой секундите.  
    seconds = 0 # Началната стойност на брояча е нула секунди.  
    while True: # Повтаряме и тази задача безкрайно.  
        seconds += 1 # Увеличаваме брояча с една секунда.  
        print("Изминали секунди:", seconds) # Показваме колко време е минало от старта  
        .  
        await asyncio.sleep(1) # Чакаме точно една секунда без блокиране на event loop  
-а.
```

Тези примери показват две идеи:

- `asyncio` като кооперативен scheduler
- разделяне на поведението на независими задачи

Файлове в главата:

- `examples/01_blink_async.py`
- `examples/02_two_tasks.py`

Резюме

- Главата въвежда кооперативното мислене с `asyncio`.
- Показва се как две задачи могат да споделят процесора без взаимно блокиране.
- Това е основа за неблокиращи приложения върху борда.

Самопроверка

1. Какво прави `await` в асинхронната задача?
2. Защо една `async` задача не бива да работи без да отстъпва управлението?
3. Каква е практическата разлика между две `async` задачи и един блокиращ цикъл?

Упражнения

1. Добавете трета `async` задача с различен период.
2. Променете периодите на задачите и наблюдавайте как се преплитат.
3. Добавете брояч, който спира демонстрацията след определено време.

Ако не работи

- Симптом: виждате поведение само от една задача. Проверка: търсете липсващ `await` или неправилно създаване на задачите.
- Симптом: програмата не тръгва асинхронно. Проверка: уверете се, че `event loop`-ът се стартира по правилния начин.

Нататък

След това мини към следващата глава, „Променливи и константи“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част II: Езикови основи на MicroPython

Глава 3. Променливи и константи

Пример:

```
from micropython import const # Импортираме const, за да дефинираме истински MicroPython константи.
```

```
BOARD_NAME = "VK_RA4M2" # Това е обикновена низова променлива с името на платката.
```

```
LED_ALIAS = "LED1" # Това е още една обикновена променлива, която пази board alias на LED-a.
```

```
BLINK_COUNT = const(3) # Това е константа, която няма да променяме по време на работа.
```

```
BLINK_DELAY_MS = const(200) # Това е константа за закъснение в милисекунди.
```

```
uses_asyncio = True # Булева променлива ни показва дали платката има поддръжка на asyncio.
```

Тук разграничението е просто:

- променлива е нещо, което променяме
- `const()` е именувана константа, с която описваме числа и режими

Файл:

- `examples/20_language_basics/02_variables_and_constants.py`

Резюме

- Главата въвежда темата „Променливи и константи“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Променливи и константи“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Оператори, условия и цикли“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 4. Оператори, условия и цикли

Тази глава покрива аритметика, сравнения, логика и контрол на потока.

Пример:

```
left_value = 7 # Първата числова променлива участва в аритметичните операции.
right_value = 3 # Втората числова променлива е вторият операнд.

sum_value = left_value + right_value # Събираме двете стойности с оператора плюс.
logic_result = (left_value > right_value) and (left_value != right_value) # Комбинирам
е логически оператори.

if left_value < right_value: # Проверяваме първото условие.
    print("По-малка") # Този ред би се изпълнил, ако първото условие е вярно.
elif left_value == right_value: # Ако първото условие е невярно, проверяваме второто.
    print("Равна") # Този ред би се изпълнил при равенство.
else: # Ако нито едно от горните условия не е изпълнено, влизаме в else клона.
    print("По-голяма") # Тук попадаме с текущите примерни стойности.
```

Циклите са показани и в трите практически форми:

```
for index in range(3): # Изпълняваме цикъл точно три пъти.
    print(index) # Печатаме текущата стойност на брояча.

counter = 0 # Създаваме брояч за while цикъла.
while counter < 3: # Повтаряме докато броячът е по-малък от три.
    print(counter) # Печатаме текущата стойност на брояча.
    counter += 1 # Увеличаваме брояча, за да не останем в безкраен цикъл.

attempt = 0 # Създаваме брояч за демонстрацията.
while True: # Влизаме в безкраен цикъл, който ще прекъснем с break.
    print(attempt) # Печатаме текущата стойност, като това винаги се случва поне веднъж.
    attempt += 1 # Увеличаваме брояча след изпълнението на тялото.
```

```
if attempt >= 2: # Проверяваме условието за прекратяване след тялото на цикъла.  
    break # Напускаме цикъла, което прави конструкцията еквивалент на do-while.
```

Важни уточнения за начинаещи:

- goto няма в Python и в MicroPython
- switch-case няма в класическия MicroPython синтаксис на тези примери
- еквивалентите са if/elif/else, таблици, break, continue, return
- do-while се показва като while True + break

Файл:

- examples/20_language_basics/03_operators_conditions_loops.py

Резюме

- Главата въвежда темата „Оператори, условия и цикли“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Оператори, условия и цикли“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Enums в MicroPython“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 5. Enums в MicroPython

Пример:

```
from micropython import const # Импортираме const, за да правим именувани константи като прост MicroPython еквивалент на enum.
```

```
STATE_OFF = const(0) # Тази константа означава състояние "изключено".
```

```
STATE_ON = const(1) # Тази константа означава състояние "включено".
```

```
STATE_BLINK = const(2) # Тази константа означава състояние "мигане".
```

Това е правилният стил за тези уроци:

- именувани константи
- таблица за превод към четимо име

Файл:

- `examples/20_language_basics/04_enums.py`

Резюме

- Главата въвежда темата „Enums в MicroPython“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Enums в MicroPython“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Масиви и таблици“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 6. Масиви и таблици

Пример:

```
from array import array # Импортираме array, за да покажем по-компактен числов масив от list.

adc_samples = [1200, 1215, 1198, 1222] # Това е обикновен list, който играе роля на масив от измервания.

led_table = [["LED1", "P204"], ["SW1", "P400"]] # Това е вложен масив, тоест таблица от редове и колони.

pwm_duty_values = array("H", [0, 16384, 32768, 49152, 65535]) # Тук пазим 16-битови стойности в компактен масив.

raw_bytes = bytearray([0x10, 0x20, 0x30, 0x40]) # bytearray е удобен за сурови байтове и комуникационни буфери.
```

Тук има четири различни вида колекции:

- list
- вложен list, тоест масив от масиви
- array
- bytearray

Файл:

- examples/20_language_basics/05_arrays.py

Резюме

- Главата въвежда темата „Масиви и таблици“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Масиви и таблици“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Структури в MicroPython“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 7. Структури в MicroPython

Вместо `struct` от C, в MicroPython се ползват:

- `dict`
- `namedtuple`
- клас
- списък от такива обекти

Пример:

```
from collections import namedtuple # Импортираме namedtuple като лек и четим заместите  
л на проста структура.
```

```
SensorRecord = namedtuple("SensorRecord", ("name", "pin", "unit")) # Описваме поле по  
поле една проста структура.
```

```
light_sensor = {"name": "Light", "adc_pin": "P000", "limits": {"dark": 800, "bright": 3  
000}} # dict е най-честият MicroPython заместител на структура.
```

```
devices = [{"name": "LED1", "pin": "P204", "kind": "output"}] # Тук правим списък от с  
труктури, както в C бихме имали масив от struct.
```

Тук има и:

- структура от структури чрез вложен `dict`
- масив от структури чрез списък от `dict`

Файл:

- `examples/20_language_basics/06_structures.py`

Резюме

- Главата въвежда темата „Структури в MicroPython“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Структури в MicroPython“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.

3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Указатели и еквивалентите им в MicroPython“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 8. Указатели и еквивалентите им в MicroPython

Тази глава е умишлено преведена към MicroPython, а не към C синтаксис.

Пример:

```
adc_list = [100, 200, 300] # Създаваме изменяем обект, който ще играе ролята на споделени данни.  
alias_list = adc_list # Тук не се копира list, а и двете имена сочат към един и същ обект.  
alias_list[1] = 999 # Промяната през второто име се вижда и през първото име.  
  
tx_buffer = bytearray([1, 2, 3, 4]) # bytearray е полезен за буфери, които се редактират "на място".  
tx_view = memoryview(tx_buffer) # memoryview дава прозорец към същите байтове без копияне.  
tx_view[2] = 77 # Промяната през memoryview променя и оригиналния буфер.
```

Тук се показват:

- две имена към един и същ обект
- промяна на буфер „на място“
- memoryview като евтин прозорец към същите данни
- ниско ниво чрез machine.mem8, mem16, mem32

Файлове:

- examples/20_language_basics/07_pointers_equivalents.py
- examples/28_machine_misc/02_machine_mem_and_bootloader.py

Резюме

- Главата въвежда темата „Указатели и еквивалентите им в MicroPython“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.

- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Указатели и еквивалентите им в MicroPython“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Функции, callbacks и таблици от функции“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 9. Функции, callbacks и таблици от функции

Пример:

```
def add_values(left_value, right_value): # Дефинираме функция, която получава два вход
ни параметъра.

    return left_value + right_value # Връщаме резултата като изходна стойност.

def run_callback(callback, value): # Дефинираме функция, която приема друга функция ка
то параметър.

    callback(value) # Извикваме подадената функция с подадената стойност.

def square(value): # Това е първата функция за таблицата от функции.

    return value * value # Връщаме квадрата на аргумента.

operation_table = [square] # Това е списък от функции, тоест таблица от извикваеми обе
кти.
```

Тази глава покрива:

- входни параметри
- връщане на стойности
- multiple return values
- mutable параметри вместо „предаване с указател“

- функция като стойност
- списък от функции като таблица от „function pointers“

Файл:

- `examples/20_language_basics/08_functions.py`

Резюме

- Главата въвежда темата „Функции, callbacks и таблици от функции“ чрез кратък изпълним MicroPython пример.
- Идеята е показана в чист синтаксис, без да се губи връзката с бъдещите хардуерни примери.
- Това е езиков строителен блок за по-сложните глави нататък.

Самопроверка

1. Коя е централната идея в „Функции, callbacks и таблици от функции“?
2. Кои конструкции от примера трябва да разпознавате без колебание?
3. Как бихте приложили тази идея в по-реален бордов пример?

Упражнения

1. Променете една стойност, условие или име в примера и проследете ефекта.
2. Добавете втори кратък пример по същата тема с други данни.
3. Свържете техниката от тази глава с материала от предишната глава.

Ако не работи

- Симптом: резултатът е различен от очакваното. Проверка: прегледайте отстъпите, имената и типа на използваните стойности.
- Симптом: логиката е неясна. Проверка: отпечатвайте междинните стойности и следете реда на изпълнение.

Нататък

След това мини към следващата глава, „Цифров изход“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част III: Цифрови и аналогови входове и изходи

Глава 10. Цифров изход

Пример:

```
from machine import Pin # Импортираме класа Pin, за да използваме LED1 като цифров изход.

import time # Импортираме time, за да правим видими паузи между смените на логическото ниво.

led = Pin("LED1", Pin.OUT, value=1) # Настройваме LED1 като изход и го оставяме изгасен в началото.
```

```
led.value(0) # Подаваме логическа нула на изхода.  
time.sleep_ms(700) # Изчакваме 700 ms, за да се види светването.  
led.value(1) # Подаваме логическа единица на изхода.
```

Тук се обясняват:

- логическа 0
- логическа 1
- защо при active-low LED те не означават директно „изключено“ и „включено“
- че допустимият ток и товароспособността се гледат по hardware manual и схемата на борда

Файлове:

- examples/21_digital_io/01_digital_output.py
- examples/01_blink_async.py

Резюме

- Главата показва как темата „Цифров изход“ се връзва към реален хардуерен ресурс на платката.
- Основният фокус е върху правилния API, настройката на пиновете и наблюдаемия резултат.
- Тук езиковият синтаксис вече служи на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Кой хардуерен ресурс или API е централният в „Цифров изход“?
2. Коя стойност, режим или параметър влияе най-много на поведението?
3. Как бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и наблюдавайте как се променя физическият ефект.
2. Добавете кратък печат за диагностика на текущото състояние или измерване.
3. Комбинирайте темата с LED1 или SW1, ако това е приложимо.

Ако не работи

- Симптом: няма хардуерна реакция. Проверка: сверете alias/pin, режима и активното логическо ниво.
- Симптом: реакцията е обърната или шумна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

След това мини към следващата глава, „Цифров вход“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 11. Цифров вход

Пример:

```
from machine import Pin # Импортираме Pin, за да четем бутона SW1 като цифров вход.  
  
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме бутона като вход с вътрешен pull-up резистор.  
  
print(button.value()) # Вземаме текущата сурова логическа стойност на входа.
```

Тук се обясняват:

- входно ниво
- активен нисък бутон
- вътрешен PULL_UP
- защо PULL_DOWN не е част от тези уроци за този порт

Файл:

- `examples/21_digital_io/02_digital_input.py`

Резюме

- Главата показва как темата „Цифров вход“ се връзва към реален хардуерен ресурс на платката.
- Основният фокус е върху правилния API, настройката на пиновете и наблюдаемия резултат.
- Тук езиковият синтаксис вече служи на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Кой хардуерен ресурс или API е централният в „Цифров вход“?
2. Коя стойност, режим или параметър влияе най-много на поведението?
3. Как бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и наблюдавайте как се променя физическият ефект.
2. Добавете кратък печат за диагностика на текущото състояние или измерване.
3. Комбинирайте темата с LED1 или SW1, ако това е приложимо.

Ако не работи

- Симптом: няма хардуерна реакция. Проверка: сверете `alias/pin`, режима и активното логическо ниво.
- Симптом: реакцията е обърната или шумна. Проверка: прегледайте `active-low` особености, `pull-up` конфигурация и свързването.

Нататък

След това мини към следващата глава, „Аналогов вход и ADC“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 12. Аналогов вход и ADC

Пример:

```
from machine import ADC # Импортираме класа ADC, чрез който се четат аналоговите канал и.

temperature_adc = ADC(ADC.CORE_TEMP) # Създаваме ADC обект за вътрешния температурен канал.

vref_adc = ADC(ADC.CORE_VREF) # Създаваме ADC обект за вътрешното опорно напрежение.

print(temperature_adc.read()) # Четем суровата ADC стойност на вътрешния температурен канал.

print(vref_adc.read()) # Четем суровата ADC стойност на вътрешното опорно напрежение.
```

Тук се учат:

- какво е ADC
- как напрежението става число
- разликата между сурово четене и read_u16()
- защо вътрешните канали са добър старт

Важно уточнение за RA4M2:

- Този MCU е с ADC12, тоест хардуерната резолюция тук е 12 bit, а поддържаните режими са 12 / 10 / 8 bit.
- В текущия порт може да се избира режим още при създаване на обекта: ADC(pin_or_channel, bits=8), ADC(..., bits=10), ADC(..., bits=12).
- Настройката за bits е глобална за ADC периферията, а не отделна за всеки ADC обект.

Кратък пример:

```
from machine import ADC # Импортираме ADC за избор на резолюция при създаване на обект а.

adc_fast = ADC("P000", bits=8) # Създаваме ADC обект с 8-битова резолюция за бързо четене.

adc_balanced = ADC("P001", bits=10) # Създаваме ADC обект с 10-битова резолюция за баланс бързина/точност.

adc_precise = ADC("P002", bits=12) # Създаваме ADC обект с 12-битова резолюция за максимална точност.

print(adc_fast.read(), adc_balanced.read(), adc_precise.read()) # Четем и трите канала и показваме резултатите.
```

Практическа бележка за времето на преобразуване при текущите тактове на VK_RA4M2:

- при минимално валидно семплиране и ADCLK = 50 MHz самото A/D преобразуване е приблизително 0.31 us за 8 bit, 0.35 us за 10 bit, 0.39 us за 12 bit

- при по-дълго семплиране, например около reset/default настройката на ADSSTR, стойностите са приблизително 0.41 us, 0.45 us, 0.49 us
- това е време на хардуерния ADC, а не пълното време на извикването ADC.read() от Python
- ако искате да мерите целия път през MicroPython, използвайте time.ticks_us() около самото извикване
- за практическо измерване на пълното време на ADC.read() виж examples/23_timing/07_adc_timing_measure.py, където се мери последователно в 8 / 10 / 12 bit

Наличните външни ADC пинове в този курс са:

P000, P001, P002, P003, P004, P005, P006, P007, P008, P013, P014, P015, P500

Файл:

- examples/22_analog/01_analog_input_adc.py
- examples/23_timing/07_adc_timing_measure.py

Резюме

- Главата показва как темата „Аналогов вход и ADC“ се връзва към реален хардуерен ресурс на платката.
- Основният фокус е върху правилния API, настройката на пиновете и наблюдаемия резултат.
- Тук езиковият синтаксис вече служи на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Кой хардуерен ресурс или API е централният в „Аналогов вход и ADC“?
2. Коя стойност, режим или параметър влияе най-много на поведението?
3. Как бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и наблюдавайте как се променя физическият ефект.
2. Добавете кратък печат за диагностика на текущото състояние или измерване.
3. Комбинирайте темата с LED1 или SW1, ако това е приложимо.

Ако не работи

- Симптом: няма хардуерна реакция. Проверка: сверете alias/pin, режима и активното логическо ниво.
- Симптом: реакцията е обърната или шумна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

След това мини към следващата глава, „PWM“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 13. PWM

Базов пример:

```
from machine import Pin, PWM # Импортираме Pin и PWM, за да генерираме PWM сигнал върху валиден PWM пин.
```

```
pwm = PWM(Pin("P107"), freq=1000, duty=50) # Създаваме PWM обект с честота 1 kHz и duty 50 процента.
```

Разширен пример за общ GPT канал:

```
pwm_a = PWM(Pin("P107"), freq=1000, duty=25) # Създаваме PWM върху изход A с честота 1 kHz и duty 25 процента.
```

```
pwm_b = PWM(Pin("P106"), freq=1000, duty=75) # Създаваме PWM върху изход B със същата честота, но с duty 75 процента.
```

```
pwm_a.freq(2000) # Променяме честотата от страна на изход A, което ще промени и канала B, защото са върху един GPT канал.
```

Тук се вижда, че:

- duty може да е различен
- честотата се споделя от двата изхода на общия GPT канал
- на този порт PWM е върху GPT периферията и общият канал налага обща freq() за A/B изходите
- проверката на реалната форма, фронтове и jitter става с осцилоскоп или логически анализатор, а не само по print от Python

Файлове:

- examples/22_analog/02_pwm_output.py
- examples/03_pwm_ab_same_channel.py

Резюме

- Главата показва как темата „PWM“ се връзва към реален хардуерен ресурс на платката.
- Основният фокус е върху правилния API, настройката на пиновете и наблюдаемия резултат.
- Тук езиковият синтаксис вече служи на физическо поведение, а не само на абстрактен пример.

Самопроверка

1. Кой хардуерен ресурс или API е централният в „PWM“?
2. Коя стойност, режим или параметър влияе най-много на поведението?
3. Как бихте проверили на реален борд, че примерът работи коректно?

Упражнения

1. Променете един основен параметър и наблюдавайте как се променя физическият ефект.
2. Добавете кратък печат за диагностика на текущото състояние или измерване.
3. Комбинирайте темата с LED1 или SW1, ако това е приложимо.

Ако не работи

- Симптом: няма хардуерна реакция. Проверка: сверете alias/pin, режима и активното логическо ниво.
- Симптом: реакцията е обърната или шумна. Проверка: прегледайте active-low особености, pull-up конфигурация и свързването.

Нататък

След това мини към следващата глава, „Закъснения, хардуерни таймери и Timer(-1)“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част IV: Таймери, прекъсвания и време

Глава 14. Закъснения, хардуерни таймери и Timer(-1)

Блокиращо закъснение:

```
import time # Импортираме time за блокиращото закъснение.  
time.sleep_ms(300) # Това закъснение блокира изпълнението на останалия код.
```

Хардуерен таймер:

```
from machine import Timer # Импортираме Timer за хардуерния таймер.  
  
timer = Timer(1) # Използваме първия от двата налични machine.Timer канала.  
timer.init(freq=4, callback=timer_callback, hard=False) # Стартираме Timer(1) на 4 Hz  
с callback извън hard IRQ контекст.
```

Софтуерен таймер:

```
from machine import Pin, Timer # Импортираме Pin и Timer за демонстрацията на софтуерн  
ия таймер.  
  
import time # Импортираме time, за да оставим таймера да работи кратко време.  
  
led = Pin("LED1", Pin.OUT) # Вземаме LED1 като видим индикатор за сработване на callba  
ck-a.  
  
soft_timer = Timer(-1) # Създаваме софтуерен таймер чрез идентификатор -1.  
soft_state = {"count": 0} # Пазим броя сработвания в mutable структура.  
started = False # С този флаг ще следим дали таймерът е стартирал успешно.  
  
def soft_timer_callback(timer_object): # Това е функцията, която ще се извиква при вся  
ко сработване на софтуерния таймер.  
    soft_state["count"] = soft_state["count"] + 1 # Увеличаваме брояча при всяко събит  
ие.  
    led.value(0 if led.value() else 1) # Превключваме LED, за да виждаме че callback-ъ  
т се изпълнява.  
  
if not started: # Ако още не сме стартирали таймера, пробваме втория стил с честота.
```

```

try: # Влизаме в защитен блок за втория опит.
    soft_timer.init(freq=4, callback=soft_timer_callback, hard=False) # Пробваме и
нициализация с честота 4 Hz.
    started = True # Отбелязваме, че вторият стил е успял.
    print("Timer(-1) тръгна с freq API.") # Печатаме кой резервен стил е сработил.
except Exception as second_error: # Ако и вторият стил не тръгне, само информираме
потребителя.
    print("freq API също не тръгна:", second_error) # Показваме причината и за вто
рия неуспех.

```

Тук трябва да остане ясно:

- Timer(1) и Timer(2) са хардуерни
- Timer(-1) е софтуерен
- asyncio е още една неблокираща алтернатива

Практическа бележка за time.ticks_us():

- ползвайте time.ticks_us() и time.ticks_diff() за относителни интервали в микросекунди
- в този порт time.ticks_us() идва от хардуерно поддържан mp_hal_ticks_us(), а не от измислен Python брояч
- това е много полезно за сравнение на latency и ADC.read() време, но не е заместител на осцилоскоп при хардуерна прецизност

Файлове:

- examples/23_timing/01_delays_and_timers.py
- examples/23_timing/04_software_timer_minus1.py
- examples/23_timing/05_timer_compare_hw_sw.py
- examples/23_timing/07_adc_timing_measure.py

Резюме

- Главата развива темата „Закъснения, хардуерни таймери и Timer(-1)“ около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Коя времева или събитийна идея е ключова в „Закъснения, хардуерни таймери и Timer(-1)“?
2. Как бихте разграничили удобна демонстрация от надеждно реално поведение?
3. Кое в тази глава би се повредило най-лесно при твърде блокиращ код?

Упражнения

1. Променете интервал, период или timeout и проследете разликата в поведението.

2. Добавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. Сравнете поведението на два различни подхода от главата върху една и съща задача.

Ако не работи

- Симптом: поведението е нестабилно във времето. Проверка: търсете дълги блокиращи участъци и неправилни единици за време.
- Симптом: получавате твърде много или твърде малко събития. Проверка: вижте прагове, debounce, периоди и условия за сработване.

Нататък

След това мини към следващата глава, „Прекъсвания“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 15. Прекъсвания

Пример:

```
from machine import Pin, disable_irq, enable_irq # Импортираме Pin, disable_irq и enable_irq за демонстрацията.

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # Настройваме SW1 като вход с pull-up.

def button_handler(pin_object): # Това е функцията, която ще се вика при прекъсване от бутона.
    print("IRQ") # Печатаме текст при всяко прекъсване.

button.irq(handler=button_handler, trigger=Pin.IRQ_FALLING, hard=False) # Регистрираме обработчик за падащ фронт на SW1.
```

Безопасно четене на споделени данни:

```
irq_state = disable_irq() # Временно забраняваме прекъсванията, за да вземем консистентно копие на брояча.

snapshot = irq_state_data["count"] # Копираме брояча, докато прекъсванията са спрени.

enable_irq(irq_state) # Възстановяваме предишното състояние на прекъсванията.
```

Това е преходът към реално събитийно програмиране.

Практическа бележка за измерване на latency:

- може да се направи полезна ориентировъчна оценка чрез два свързани GPIO пина и `time.ticks_us()`
- тази стойност включва GPIO пътя, IRQ доставката, soft callback scheduling и Python callback overhead
- затова тя не е равна на „чистата“ силициева interrupt latency от datasheet или осцилоскоп
- примерът `examples/23_timing/08_interrupt_latency_measure.py` иска jumper между изходен и входен пин

Файлове:

- `examples/23_timing/02_interrupts.py`
- `examples/23_timing/08_interrupt_latency_measure.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`

Резюме

- Главата развива темата „Прекъсвания“ около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Коя времева или събитийна идея е ключова в „Прекъсвания“?
2. Как бихте разграничили удобна демонстрация от надеждно реално поведение?
3. Кое в тази глава би се повредило най-лесно при твърде блокиращ код?

Упражнения

1. Променете интервал, период или `timeout` и проследете разликата в поведението.
2. Добавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. Сравнете поведението на два различни подхода от главата върху една и съща задача.

Ако не работи

- Симптом: поведението е нестабилно във времето. Проверка: търсете дълги блокиращи участъци и неправилни единици за време.
- Симптом: получавате твърде много или твърде малко събития. Проверка: вижте прагове, `debounce`, периоди и условия за сработване.

Нататък

След това мини към следващата глава, „Дебаунс на бутон“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 16. Дебаунс на бутон

Пример с `polling`:

```
if current_value != last_stable_value: # Проверяваме дали е настъпила промяна спрямо п
    oследното стабилно състояние.
    now_ms = time.ticks_ms() # Вземаме текущото време само при промяна.
    if time.ticks_diff(now_ms, last_change_ms) >= 40: # Приемаме промяната за валидна
    само ако е минало поне 40 ms.
        last_change_ms = now_ms # Обновяваме момента на последната приета промяна.
        last_stable_value = current_value # Приемаме новата стойност за стабилна.
```

Пример с `uasyncio`:

```
if stable_value == 0 and stable_count == 4: # Ако видим четири поредни проби с 0, прие  
маме валидно натискане.  
    async_presses += 1 # Увеличаваме броя на валидните асинхронни натискания.
```

Тук има два подхода:

- времеви прозорец в polling цикъл
- последователни стабилни проби в async задача

Файл:

- `examples/23_timing/03_button_debounce.py`

Резюме

- Главата развива темата „Дебаунс на бутон“ около време, събития и реакция на системата.
- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Коя времева или събитийна идея е ключова в „Дебаунс на бутон“?
2. Как бихте разграничили удобна демонстрация от надеждно реално поведение?
3. Кое в тази глава би се повредило най-лесно при твърде блокиращ код?

Упражнения

1. Променете интервал, период или `timeout` и проследете разликата в поведението.
2. Добавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. Сравнете поведението на два различни подхода от главата върху една и съща задача.

Ако не работи

- Симптом: поведението е нестабилно във времето. Проверка: търсете дълги блокиращи участъци и неправилни единици за време.
- Симптом: получавате твърде много или твърде малко събития. Проверка: вижте прагове, `debounce`, периоди и условия за сработване.

Нататък

След това мини към следващата глава, „Четене на бутон: `polling`, `IRQ`, `Timer(-1)`, `debounce` и събития“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 17. Четене на бутон: `polling`, `IRQ`, `Timer(-1)`, `debounce` и събития

Тази глава събира на едно място реалните начини за работа с бутон върху `VK_RA4M2`.

Суровото четене на `SW1` е само началото.

Практическият въпрос е как да преведем шумния active-low вход към стабилни събития като:

- PRESS
- RELEASE
- CLICK
- LONG_PRESS
- HOLD_START
- HOLD_REPEAT

Първата идея е проста: `button.value()` ни дава ниво, но приложението почти винаги иска събития.

Пример с polling и преходи между стабилни състояния:

```
raw_value = button.value() # Четем суровата моментна стойност на бутона.
if raw_value != last_raw_value: # Ако видим ново сурово ниво, започваме нов debounce п
розорец.
    last_raw_value = raw_value # Запомняме новата сурова стойност.
    raw_change_ms = now_ms # Запомняме кога е започнала промяната.
if raw_value != stable_value and time.ticks_diff(now_ms, raw_change_ms) >= DEBOUNCE_MS:
    # Ако новото ниво е стабилно достатъчно дълго, приемаме ново логическо състояние.
    stable_value = raw_value # Обновяваме стабилното състояние на бутона.
    if stable_value == 0: # Ако стабилното състояние е 0, имаме валидно натискане.
        print("EVENT: PRESS") # Печатаме събитие за натискане.
    else: # Ако стабилното състояние е 1, имаме валидно отпускане.
        print("EVENT: RELEASE") # Печатаме събитие за отпускане.
```

Тук вече имаме важна промяна в начина на мислене:

- не гледаме „ниво в момента“
- гледаме „последно стабилно състояние“
- генерираме събитие само при стабилен преход

Ако искаме по-бърза реакция от чистото polling, ползваме прекъсване, но не правим тежката логика вътре в IRQ обработчика.

Пример с IRQ_FALLING | IRQ_RISING и отложена обработка:

```
def button_irq_handler(pin_object): # Това е краткият IRQ обработчик за суровите фронт
ове на бутона.
    irq_state_data["pending"] = 1 # Маркираме, че главният цикъл трябва да довърши лог
ическата обработка.
    irq_state_data["deadline_ms"] = time.ticks_add(time.ticks_ms(), DEBOUNCE_MS) # Отл
агаме окончателното решение след debounce интервала.
```

```

if pending and time.ticks_diff(now_ms, deadline_ms) >= 0: # Ако има чакаща работа и de
bounce прозорецът е изтекъл, вземаме стабилно решение.

    stable_candidate = button.value() # Четем реалното ниво след краткото изчакване.

    if stable_candidate != previous_stable_value: # Ако стабилното ниво реално е ново,
генерираме логическо събитие.

        print("EVENT: PRESS" if stable_candidate == 0 else "EVENT: RELEASE") # Печатам
е кое логическо събитие е настъпило.

```

Това е по-добрият стил за бутон по прекъсване:

- IRQ само събужда логиката
- debounce се решава извън ISR
- събитието пак е стабилен PRESS или RELEASE, а не суров фронт

Третият подход е Timer(-1) като периодичен sampler.

Той е полезен, когато искаме еднакъв ритъм на пробовземане и по-богати събития за задържане.

Пример със софтуерен Timer(-1) и hold логика:

```

soft_timer.init(period=SAMPLE_MS, mode=Timer.PERIODIC, callback=button_sample_callback)
# Стартираме Timer(-1) като периодичен sampler на бутона.

if timer_state["history"] == 0x00 and timer_state["stable_value"] != 0: # Ако последни
те осем проби са 0, приемаме стабилно натискане.

    timer_state["event_code"] = EVENT_PRESS # Записваме PRESS събитие за основния цикъ
л.

elif timer_state["stable_value"] == 0 and not timer_state["hold_started"] and time.tick
s_diff(now_ms, timer_state["press_start_ms"]) >= LONG_PRESS_MS: # Ако бутонът е натисн
ат достатъчно дълго, излъчваме начало на задържане.

    timer_state["event_code"] = EVENT_HOLD_START # Записваме HOLD_START събитие.

elif timer_state["stable_value"] == 0 and timer_state["hold_started"] and time.ticks_di
ff(now_ms, timer_state["next_repeat_ms"]) >= 0: # Ако вече сме в задържане и е дошло в
реме за повторение, излъчваме repeat.

    timer_state["event_code"] = EVENT_HOLD_REPEAT # Записваме HOLD_REPEAT събитие.

```

Тук се появяват вече различни класове събития:

- PRESS при стабилен преход 1 -> 0
- RELEASE при стабилен преход 0 -> 1
- CLICK при кратко натискане и отпускане
- LONG_PRESS при достигане на праг по време на задържане
- HOLD_START при началото на продължително задържане
- HOLD_REPEAT при периодично повтаряне, докато бутонът остава натиснат

Четвъртият подход е debounce чрез битово изместване.

Той е компактен и много удобен за периодичен sampler, защото свива последните N проби в една история.

Пример с shift-register debounce:

```
history = ((history << 1) | button.value()) & 0xFF # Премества историята наляво и до
бавяме новата проба като най-млад бит.

if history == 0x00 and stable_value != 0: # Ако последните осем проби са натиснати, пр
иемаме стабилно PRESS събитие.

    print("EVENT: PRESS") # Печатаме събитие за натискане.

elif history == 0xFF and stable_value != 1: # Ако последните осем проби са отпуснати,
приемаме стабилно RELEASE събитие.

    print("EVENT: RELEASE") # Печатаме събитие за отпускане.
```

Тук смисълът на history е директен:

- 0x00 означава осем поредни проби с натиснат бутон
- 0xFF означава осем поредни проби с отпуснат бутон
- всички междинни стойности означават „още не е стабилно“

Практически избор на подход:

- ако искате най-простия старт, започнете с polling
- ако искате бързо събуждане на логиката, използвайте IRQ с отложена обработка
- ако искате равномерно пробовземане и hold/repeat семантика, използвайте Timer(-1)
- ако искате компактен debounce филтър, използвайте битово изместване

Файлове:

- examples/23_timing/09_button_polling_events.py
- examples/23_timing/10_button_irq_deferred.py
- examples/23_timing/11_button_soft_timer_hold.py
- examples/23_timing/12_button_shift_debounce.py
- examples/23_timing/02_interrupts.py
- examples/23_timing/03_button_debounce.py
- examples/23_timing/04_software_timer_minus1.py

Резюме

- Главата събира четенето на бутон като ниво, като стабилен преход и като събитийна система.
- Показани са polling, IRQ с отложена обработка, Timer(-1) sampler и debounce чрез битово изместване.
- Основната идея е, че приложението почти винаги иска PRESS, RELEASE и HOLD събития, а не суров шумен вход.

Самопроверка

1. Каква е разликата между суров фронт от IRQ и стабилно логическо събитие PRESS?
2. Кога `Timer(-1)` е по-подходящ от чист polling цикъл за бутон?
3. Какво означават `0x00` и `0xFF` при debounce чрез битово изместване?

Упражнения

1. Разширете `09_button_polling_events.py`, така че след `LONG_PRESS` да не печата и `CLICK` при отпускане.
2. Добавете `CLICK` и към `10_button_irq_deferred.py`, като използвате продължителността между `PRESS` и `RELEASE`.
3. Променете размера на debounce историята в `12_button_shift_debounce.py` от 8 на 4 проби и сравнете устойчивостта.

Ако не работи

- Симптом: получавате много `PRESS` и `RELEASE` събития от едно натискане. Проверка: debounce прозорецът или пробовземането са твърде къси.
- Симптом: `LONG_PRESS` или `HOLD_REPEAT` не идват. Проверка: вижте дали държите бутона достатъчно дълго и дали прагът за задържане не е твърде голям.

Нататък

След това мини към следващата глава, „RTC“, за да надградиш текущата идея в естествената й следваща стъпка.

Глава 18. RTC

Пример:

```
from machine import RTC # Импортираме RTC, за да работим с часовника в реално време.

rtc = RTC() # Създаваме обект за достъп до вградения RTC модул.
print(rtc.info()) # Печатаме диагностичната информация за стартиране на RTC.
print(rtc.calibration()) # Печатаме текущата калибрационна стойност.
print(rtc.datetime()) # Четем текущата дата и час като осемелементен tuple.
```

Тази глава покрива:

- диагностика на RTC
- current datetime tuple
- безопасно задаване на време зад флаг

Файл:

- `examples/23_timing/06_rtc_basic.py`

Резюме

- Главата развива темата „RTC“ около време, събития и реакция на системата.

- Показва се кога простото линейно мислене вече не е достатъчно и трябва по-стриктно управление на времето.
- Това е преход към по-реактивен и по-системен стил на програмиране.

Самопроверка

1. Коя времева или събитийна идея е ключова в „RTC“?
2. Как бихте разграничили удобна демонстрация от надеждно реално поведение?
3. Кое в тази глава би се повредило най-лесно при твърде блокиращ код?

Упражнения

1. Променете интервал, период или timeout и проследете разликата в поведението.
2. Добавете кратка диагностична индикация с LED1 или печат през сериен терминал.
3. Сравнете поведението на два различни подхода от главата върху една и съща задача.

Ако не работи

- Симптом: поведението е нестабилно във времето. Проверка: търсете дълги блокиращи участъци и неправилни единици за време.
- Симптом: получавате твърде много или твърде малко събития. Проверка: вижте прагове, debounce, периоди и условия за сработване.

Нататък

След това мини към следващата глава, „I2C master и runtime избор на пинове“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част V: Сериен интерфейс

Глава 19. I2C master и runtime избор на пинове

Базов пример:

```
from machine import I2C # Импортираме I2C за работа като master върху наличния хардуерен контролер.
```

```
i2c = I2C(1, freq=100000) # Създаваме I2C(1) на 100 kHz върху бордовите пинове P100 и P101.
```

```
print(i2c.scan()) # Сканираме шината за устройства с 7-битов адрес.
```

Runtime пинове:

```
from machine import I2C, Pin # Импортираме I2C и Pin, за да покажем runtime избиране на SCL и SDA.
```

```
i2c = I2C(1, freq=400000, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2C(1) на 400 kHz с изрично зададени пинове.
```

Този глава е важна и заради конфликтите:

- I2C(0) ползва P400/P401
- P400 е и SW1

Файлове:

- examples/24_i2c/01_i2c_master_basic.py
- examples/24_i2c/02_i2c_runtime_pins.py

Резюме

- Главата въвежда интерфейса „I2C master и runtime избор на пинове“ като реален комуникационен канал с външен свят.
- Фокусът е върху линиите, ролите на участниците и минималния практически тест.
- Тук правилната връзка между софтуер и окабеляване е толкова важна, колкото самият код.

Самопроверка

1. Коя е основната роля на интерфейса в „I2C master и runtime избор на пинове“?
2. Кой параметър или сигнал е най-критичен за първоначален успех на примера?
3. Как бихте валидирали комуникацията преди да включите реално външно устройство?

Упражнения

1. Направете минимален диагностичен тест и отпечатайте суровия резултат.
2. Променете конфигурационен параметър и вижте как това влияе на комуникацията.
3. Опишете с кратък коментар кои линии или роли в примера отговарят на всяка физическа връзка.

Ако не работи

- Симптом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нужните pull-up или control сигнали.
- Симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

След това мини към следващата глава, „SoftI2C и I2CTarget“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 20. SoftI2C и I2CTarget

SoftI2C:

```
from machine import SoftI2C, Pin # Импортираме SoftI2C и Pin за софтуерна I2C шина върху GPIO.
```

```
soft_i2c = SoftI2C(scl=Pin("P105", Pin.OPEN_DRAIN), sda=Pin("P104", Pin.OPEN_DRAIN), freq=100000) # Създаваме SoftI2C на 100 kHz.
```

I2CTarget:

```
from machine import I2CTarget, Pin # Импортираме I2CTarget и Pin за работа в target ил и slave режим.
```

```
memory_buffer = bytearray(range(16)) # Подготвяме малък memory-backed буфер, който target-ът ще обслужва.
```

```
target = I2CTarget(1, addr=0x42, mem=memory_buffer, mem_addrsize=8, scl=Pin("P100"), sda=Pin("P101")) # Създаваме I2CTarget(1) върху стандартните пинове на канал 1.
```

Тук се вижда:

- софтуерна I2C шина върху GPIO
- целево устройство с memory-backed буфер

Файлове:

- `examples/24_i2c/03_softi2c_basic.py`
- `examples/24_i2c/04_i2ctarget_memory.py`

Резюме

- Главата въвежда интерфейса „SoftI2C и I2CTarget“ като реален комуникационен канал с външен свят.
- Фокусът е върху линиите, ролите на участниците и минималния практически тест.
- Тук правилната връзка между софтуер и окабеляване е толкова важна, колкото самият код.

Самопроверка

1. Коя е основната роля на интерфейса в „SoftI2C и I2CTarget“?
2. Кой параметър или сигнал е най-критичен за първоначален успех на примера?
3. Как бихте валидирали комуникацията преди да включите реално външно устройство?

Упражнения

1. Направете минимален диагностичен тест и отпечатайте суровия резултат.
2. Променете конфигурационен параметър и вижте как това влияе на комуникацията.
3. Опишете с кратък коментар кои линии или роли в примера отговарят на всяка физическа връзка.

Ако не работи

- Симптом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нужните pull-up или control сигнали.
- Симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

След това мини към следващата глава, „Hardware SPI и SoftSPI“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 21. Hardware SPI и SoftSPI

Hardware SPI:

```
from machine import SPI, Pin # Импортираме SPI и Pin за достъп до хардуерния SPI канал
.

spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P101"), miso=Pin("P100"), cs=Pin("P103")) # Създаваме SPI обект върху валидните board пинове.
```

SoftSPI:

```
from machine import SoftSPI, Pin # Импортираме SoftSPI и Pin за софтуерен SPI върху GPIO.

soft_spi = SoftSPI(baudrate=100000, polarity=0, phase=0, sck=Pin("P105"), mosi=Pin("P104"), miso=Pin("P106")) # Създаваме софтуерен SPI обект върху три GPIO пина.
```

В тази книга SPI остава 1 канал.

Файлове:

- examples/25_spi/01_spi_basic.py
- examples/25_spi/02_softspi_basic.py

Резюме

- Главата въвежда интерфейса „Hardware SPI и SoftSPI“ като реален комуникационен канал с външен свят.
- Фокусът е върху линиите, ролите на участниците и минималния практически тест.
- Тук правилната връзка между софтуер и окабеляване е толкова важна, колкото самият код.

Самопроверка

1. Коя е основната роля на интерфейса в „Hardware SPI и SoftSPI“?
2. Кой параметър или сигнал е най-критичен за първоначален успех на примера?
3. Как бихте валидирали комуникацията преди да включите реално външно устройство?

Упражнения

1. Направете минимален диагностичен тест и отпечатайте суровия резултат.
2. Променете конфигурационен параметър и вижте как това влияе на комуникацията.
3. Опишете с кратък коментар кои линии или роли в примера отговарят на всяка физическа връзка.

Ако не работи

- Симптом: няма отговор от шината. Проверка: първо сверете пиновете, адреса/ролята и нужните pull-up или control сигнали.
- Симптом: получавате нестабилни или безсмислени данни. Проверка: намалете скоростта и проверете окабеляването стъпка по стъпка.

Нататък

След това мини към следващата глава, „TouchPad: базово четене“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част VI: TouchPad, памет и системни функции

Глава 22. TouchPad: базово четене

Пример:

```
from machine import Pin, TouchPad # Импортираме Pin и TouchPad за работа с CTSU вход.

touch = TouchPad(Pin("P205")) # Създаваме TouchPad върху валиден touch пин P205.
touch.config(500) # Задаваме праг 500 за метода value().
print(touch.read()) # Четем необработената CTSU стойност.
print(touch.value()) # Преобразуваме стойността към 0 или 1 според прага.
print(touch.read_value()) # Взимаме едновременно count, value и последна FSP грешка.
```

Тук се виждат:

- сурова стойност
- логическа стойност по праг
- комбинирано четене

Файл:

- `examples/26_touchpad/01_touchpad_basic.py`

Резюме

- Главата развива темата „TouchPad: базово четене“ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограниченията и режима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Коя е най-важната особеност или ограничение в „TouchPad: базово четене“?
2. Кой ресурс е `volatile` и кой е `persistent` в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един ключов праг, буфер или режим и сравнете поведението преди и след това.

3. Свържете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- Симптом: поведението е чувствително или шумно. Проверка: търсете грешни прагове, неподходящ режим или неправилни очаквания към хардуера/порта.
- Симптом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеширане и порт-специфични особености.

Нататък

След това мини към следващата глава, „TouchPad: диагностика, cached API и асинхронно следене“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 23. TouchPad: диагностика, cached API и асинхронно следене

Диагностичен пример:

```
TouchPad.sample_rate(20) # Стартираме глобалния cooperative sampler на 20 пълни scan-a в секунда.
touch.start() # Стартираме non-blocking scan, ако не е в ход такъв.
TouchPad.service() # Придвижваме вътрешния sampler една стъпка от VM контекст.
print(touch.ready(), touch.read_cached(), touch.value_cached()) # Показваме cached състоянието.
print(touch.diagnose(8)) # Печатаме резултат от кратък диагностичен цикъл.
```

Асинхронен пример:

```
TouchPad.sample_rate(50) # Стартираме sampler на 50 scan-a в секунда.
async def touch_task(): # Асинхронна задача за следене на TouchPad.
    while True: # Безкрайно следене.
        if tp.ready(): # Проверяваме дали пробата е готова.
            print(tp.read_cached(), tp.value_cached(), tp.age_ms()) # Печатаме cached стойностите.
            await asyncio.sleep_ms(20) # Сканираме на 20 ms.
```

Тази глава е важна, защото показва не само „натиснато/ненатиснато“, а и сервизно поведение на CTSU слоя.

Файлове:

- examples/26_touchpad/02_touchpad_diagnostics.py
- examples/04_touchpad_async.py

Резюме

- Главата развива темата „TouchPad: диагностика, cached API и асинхронно следене“ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограничения и режима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Коя е най-важната особеност или ограничение в „TouchPad: диагностика, cached API и асинхронно следене“?
2. Кой ресурс е volatile и кой е persistent в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един ключов праг, буфер или режим и сравнете поведението преди и след това.
3. Свържете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- Симптом: поведението е чувствително или шумно. Проверка: търсете грешни прагове, неподходящ режим или неправилни очаквания към хардуера/порта.
- Симптом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеширане и порт-специфични особености.

Нататък

След това мини към следващата глава, „renesas.Flash, /flash и dataflash“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 24. renesas.Flash, /flash и dataflash

Пример с renesas.Flash:

```
import os # Импортираме os, за да покажем съдържанието на файловата система /flash.
import renesas # Импортираме renesas модула, който излага Flash block device.

flash = renesas.Flash() # Създаваме block device обект върху вътрешната flash памет.
sector = bytearray(16) # Подготвяме малък буфер за четене на първите байтове от блок 0
.
flash.readblocks(0, sector) # Чetem първите 16 байта от блок 0 в буфера.
print(os.listdir("/flash")) # Показваме кои файлове има във вътрешната файлова система
.
```

Пример с dataflash:

```
import dataflash # Импортираме dataflash модула за достъп до отделната Data Flash област.

print(dataflash.size()) # Печатаме общия размер на Data Flash областта.
print(dataflash.block_size()) # Печатаме размера на erase блока.
print(dataflash.write_size()) # Печатаме минималния размер за запис.
print(dataflash.read(0, 16)) # Четем първите 16 байта от Data Flash.
```

Пример за устойчиво състояние в dataflash и видима индикация с LED1:

```
from machine import Pin # Импортираме Pin за управление на LED1.
import dataflash # Импортираме dataflash за устойчиво съхранение на състояние.

led = Pin("LED1", Pin.OUT, value=1) # Използваме board alias-а LED1, започваме изгасен .

state = dataflash.read(0, 1)[0] # Връщаме стойността на първия байт.

if state == 0x00: # Ако предишният старт е записал 0x00, сега правим erase към 0xFF.
    dataflash.erase_block(0) # Изтриваме блок 0, всички байтове стават 0xFF.
    blinks_per_sec = 2 # Бавно мигане означава erase.
else: # Иначе приемаме, че байтът е 0xFF и записваме 0x00.
    dataflash.write(0, bytes([0x00])) # Записваме 0x00 в първия байт.
    blinks_per_sec = 10 # Бързо мигане означава write.
```

Този пример показва как един байт в dataflash може да пази състояние между два старта на програмата.

За VK_RA4M2 LED индикацията е върху LED1, защото провереният board LED е P204 и е active-low.

При всяко стартиране се редуват две състояния:

- 0xFF след erase_block(0)
- 0x00 след write(0, bytes([0x00]))

Скоростта на мигане показва кой преход е станал на текущия boot.

Тук се разграничават:

- файлова система
- block device
- отделен data flash регион

Проверени размери за текущата конфигурация на VK_RA4M2:

- MCU вариантът на тази платка е с общо 384 KB code flash и 128 KB RAM
- в текущия linker layout 290 KB са отделени за firmware/application region, а 94 KB са отделени като FLASH_FS за /flash

- отделно има 8 KB dataflash
- /flash е върху вътрешния flash block device, монтиран като FAT; block size е 512 bytes
- реално свободното място за файлове е малко по-малко от 94 KB, защото част отива за самата файлова система

Файлове:

- examples/27_storage/01_flash_blockdev.py
- examples/27_storage/02_dataflash_basic.py
- examples/27_storage/03_dataflash_state_led.py

Резюме

- Главата развива темата „renesas.Flash, /flash и dataflash“ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограниченията и режима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Коя е най-важната особеност или ограничение в „renesas.Flash, /flash и dataflash“?
2. Кой ресурс е volatile и кой е persistent в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един ключов праг, буфер или режим и сравнете поведението преди и след това.
3. Свържете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- Симптом: поведението е чувствително или шумно. Проверка: търсете грешни прагове, неподходящ режим или неправилни очаквания към хардуера/порта.
- Симптом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеширане и порт-специфични особености.

Нататък

След това мини към следващата глава, „Управление на паметта в MicroPython“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 25. Управление на паметта в MicroPython

Диагностичен пример:

```
import gc # Импортираме gc за контрол и измерване на garbage-collected heap-a.
```

```
import micropython # Импортираме micropython за mem_info и други runtime helper-и.  
  
gc.collect() # Пускаме GC в предвидим момент, за да започнем от по-чисто състояние.  
print(gc.mem_free(), gc.mem_alloc()) # Печатаме свободния и заетия heap.  
micropython.mem_info() # Печатаме обобщена информация за heap-а.
```

Буферен пример:

```
buffer = bytearray(32) # Създаваме един буфер, който ще използваме.  
window = memoryview(buffer)[8:12] # Вземаме прозорец към част от буфера без копиране.  
window[0] = 123 # Променяме първия байт през memoryview.  
print(list(buffer[8:12])) # Показваме, че оригиналният буфер е променен на място.
```

Тук „управление на паметта“ не означава ръчно местене на адреси както в C, а по-скоро:

- да разбираме кои обекти заемат място в heap-а на MicroPython
- да различаваме RAM heap от stack, firmware и постоянна flash памет
- да ограничаваме излишните алокации в чувствителни по време участъци
- да използваме буфери вместо да създаваме нови обекти във всеки цикъл

Потвърдени факти за текущия VK_RA4M2 порт:

- garbage-collected heap-ът се подава на MicroPython чрез linker символите `_heap_start` и `_heap_end`
- linker-ът заделя отделен фиксиран stack регион от 8 KB
- /flash и dataflash са постоянна памет за файлове и данни, а не RAM heap за Python обекти

Практични API-та, които реално са налични в този порт:

- `gc.collect()` за принудително стартиране на garbage collector в удобен момент
- `gc.mem_free()` и `gc.mem_alloc()` за ориентир колко heap е свободен и колко е зает
- `gc.threshold()` за по-напреднал контрол кога да се задейства GC
- `micropython.mem_info()` и `micropython.mem_info(1)` за по-подробна картина на heap-а
- `micropython.alloc_emergency_exception_buf()` за изключително важни exception-и в interrupt/critical контекст
- `micropython.heap_lock()` и `micropython.heap_unlock()` за напреднали случаи, когато временно не трябва да се допуска алокация

Практически правила, които най-често дават реален ефект:

- правете `bytearray` веднъж и после го пълнете отново, вместо да създавате нов буфер всеки път
- използвайте `memoryview`, когато искате прозорец към част от буфера без копиране

- ако имате временни големи обекти, освобождавайте ги и извиквайте `gc.collect()` в предвидим момент, а не насред критична операция
- при interrupt-и, callbacks и latency-sensitive код избягвайте нови алокации; подгответе буферите предварително
- не смесвайте `machine.mem8/mem16/mem32` с управлението на Python heap-а; това са различни нива на достъп до паметта

Тази тема се свързва естествено с вече наличните примери:

- `07_pointers_equivalents.py` показва shared reference и memoryview
- `04_i2ctarget_memory.py` показва предварително заделен mutable буфер, подаден към периферен API
- `01_flash_blockdev.py` и `02_dataflash_basic.py` показват, че persistent storage не е същото като heap RAM

Файлове:

- `examples/28_machine_misc/04_memory_management.py`
- `examples/20_language_basics/07_pointers_equivalents.py`
- `examples/24_i2c/04_i2ctarget_memory.py`
- `examples/27_storage/01_flash_blockdev.py`
- `examples/27_storage/02_dataflash_basic.py`

Резюме

- Главата развива темата „Управление на паметта в MicroPython“ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограниченията и режима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Коя е най-важната особеност или ограничение в „Управление на паметта в MicroPython“?
2. Кой ресурс е volatile и кой е persistent в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един ключов праг, буфер или режим и сравнете поведението преди и след това.
3. Свържете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- Симптом: поведението е чувствително или шумно. Проверка: търсете грешни прагове, неподходящ режим или неправилни очаквания към хардуера/порта.

- Симптом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеширане и порт-специфични особености.

Нататък

След това мини към следващата глава, „machine модулът и порт-специфично поведение“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 26. machine модулът и порт-специфично поведение

Информационен пример:

```
import machine # Импортираме machine, за да покажем системните функции на порта.

uid = machine.unique_id() # Четем уникалния 128-битов идентификатор на MCU.
uid_hex = "".join("{:02x}".format(byte_value) for byte_value in uid) # Преобразуваме байтовете до четим hex низ.

print("machine.freq() =", machine.freq()) # Показваме текущата честота на ядрото.
print("machine.unique_id() =", uid_hex) # Печатаме уникалния идентификатор в hex вид.
print("machine.reset_cause() =", machine.reset_cause()) # Печатаме причината за последния reset.

machine.info() # Печатаме подробна системна информация от самия порт.
```

Ниско ниво:

```
irq_state = machine.disable_irq() # Временно забраняваме прекъсванията и пазим предишното състояние.

machine.enable_irq(irq_state) # Възстановяваме прекъсванията веднага след демонстрацията.

print(hasattr(machine, "mem32")) # Показваме дали портът излага 32-битов memory access.
```

Sleep режими:

```
DO_LIGHTSLEEP = False # Оставяме lightsleep изключен по подразбиране.
DO_DEEPSLEEP = False # Оставяме deepsleep изключен по подразбиране.

print("Извикваме кратък machine.lightsleep(20).") # Подготвяме потребителя за кратък sleep.

machine.lightsleep(20) # Извикваме кратък lightsleep за 20 ms.

if DO_LIGHTSLEEP: # Само ако е включено, пробваме по-дълъг lightsleep.
    machine.lightsleep(100) # Извикваме lightsleep за 100 ms.
```

Тази глава е мястото за:

- machine.info

- `machine.freq`
- `machine.unique_id`
- `machine.reset_cause`
- `disable_irq` и `enable_irq`
- `mem8`, `mem16`, `mem32`
- предпазен `machine.bootloader`
- `sleep`, `lightsleep`, `deepsleep`

Практическа бележка за честоти и ток:

- `machine.freq()` е полезен за бърза проверка на runtime честотата; текущата board конфигурация е 100 MHz системен clock и 50 MHz периферни clock-и за много блокове
- токовете числа в hardware manual-а са за самия MCU при конкретни условия, а не за цялата платка през USB
- типични стойности от manual-а са около 15.4 mA в normal mode с всички peripheral clocks enabled, 6.1 mA при disabled peripheral clocks и 4.4 mA в sleep mode
- за ниска консумация manual-ът дава ориентири от 0.7 uA за software standby и приблизително 5 ... 16 uA за deep software standby според retained функциите
- по време на 12-bit A/D conversion analog current е типично около 0.8 mA, което е отделна аналогова консумация и не бива да се бърка с общата консумация на платката

Файлове:

- `examples/28_machine_misc/01_machine_info_and_id.py`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py`
- `examples/28_machine_misc/03_sleep_modes.py`

Резюме

- Главата развива темата „machine модулът и порт-специфично поведение“ около по-чувствителни, по-системни или порт-специфични възможности.
- Тук вече не е достатъчно само да извикате API-то; важно е да разбирате ограниченията и режима на работа.
- Това е зоната, в която практическата дисциплина става толкова важна, колкото и самият пример.

Самопроверка

1. Коя е най-важната особеност или ограничение в „machine модулът и порт-специфично поведение“?
2. Кой ресурс е volatile и кой е persistent в този тип пример?
3. Как бихте измерили или наблюдавали дали примерът се държи стабилно?

Упражнения

1. Добавете диагностичен печат или проста метрика, която да прави състоянието по-видимо.
2. Променете един ключов праг, буфер или режим и сравнете поведението преди и след това.

3. Свържете темата с реално действие на LED1, SW1 или малък лог в паметта, ако това е приложимо.

Ако не работи

- Симптом: поведението е чувствително или шумно. Проверка: търсете грешни прагове, неподходящ режим или неправилни очаквания към хардуера/порта.
- Симптом: резултатът не съвпада с интуицията. Проверка: разграничете RAM, постоянна памет, кеширане и порт-специфични особености.

Нататък

След това мини към следващата глава, „Динамична 7-сегментна индикация и матрично сканиране“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част VII: Индикация и входни матрици

Глава 27. Динамична 7-сегментна индикация и матрично сканиране

Пример:

```
from machine import Pin # Импортираме Pin, за да покажем базовото сканиране на външен
дисплей и бутони.

import time # Импортираме time за краткото динамично опресняване.

segment_pins = [Pin(name, Pin.OUT) for name in ("P105", "P104", "P113", "P114", "P115",
"P304", "P303")] # Това са седемте сегмента а..g.

digit_pins = [Pin(name, Pin.OUT) for name in ("P608", "P409")] # Това са двата общи из
вода за две цифри.

patterns = {"1": (0, 1, 1, 0, 0, 0, 0), "2": (1, 1, 0, 1, 1, 0, 1)} # Това са сегментн
ите шаблони за две примерни цифри.

for refresh_step in range(10): # Повтаряме кратък опреснителен цикъл десет пъти.
    for digit_index, symbol in enumerate(("1", "2")): # Показваме символ 1 на първата
цифра и 2 на втората цифра.
        for digit_pin in digit_pins: # Първо изключваме всички цифри, за да няма ghost
ing.
            digit_pin.off() # Изключваме текущата цифра.

            for segment_pin, level in zip(segment_pins, patterns[symbol]): # Зареждаме нуж
ния сегментен шаблон за текущия символ.
                segment_pin.value(level) # Настройваме сегмента според шаблона.

            digit_pins[digit_index].on() # Включваме само текущата цифра за кратко време.

            time.sleep_ms(5) # Държим цифрата включена кратко, за да се получи динамична и
ндикация.
```

Клавиатурна матрица:

```

row_pins = [Pin(name, Pin.OUT) for name in ("P408", "P600")] # Това са редовете на при-
мерна 2x2 клавиатурна матрица.

col_pins = [Pin(name, Pin.IN, Pin.PULL_UP) for name in ("P301", "P302")] # Това са кол-
оните на примерната 2x2 клавиатурна матрица.

pressed_keys = [] # Подготвяме списък за натиснати клавиши от матрицата.

for row_index, row_pin in enumerate(row_pins): # Сканираме всеки ред поотделно.
    for reset_row in row_pins: # Първо изключваме всички редове.
        reset_row.off() # Изключваме реда.
    row_pin.on() # Активираме текущия ред.
    for col_index, col_pin in enumerate(col_pins): # Проверяваме всяка колона в активн-
ия ред.
        if col_pin.value() == 0: # Ако колоната падне на 0, приемаме че бутонът е нати-
снат.
            pressed_keys.append((row_index, col_index)) # Записваме координатите на на-
тиснатия бутон.

```

Този пример изисква външен хардуер, но е много важен за свързване на GPIO, време и логика.

Файл:

- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py`

Резюме

- Главата комбинира GPIO, време и външен хардуер в една по-сложна, но много показателна демонстрация.
- Основната идея е, че мултиплексирането и матричното сканиране разчитат на правилен ред и темпо на опресняване.
- Това е добър пример как софтуерното време пряко влияе на видимия хардуерен резултат.

Самопроверка

1. Защо динамичната индикация не работи добре при твърде бавно опресняване?
2. Каква е идеята зад матричното сканиране на клавиши?
3. Кой проблем ще се появи първо, ако редът на управление на сегменти и цифри е грешен?

Упражнения

1. Променете честотата на опресняване и наблюдавайте кога започва видимо мигане.
2. Добавете проста таблица за превод от координати на матрицата към символи.
3. Използвайте LED1 като диагностичен индикатор при намерен натиснат клавиш.

Ако не работи

- Симптом: дисплеят е слаб или премигва. Проверка: периодът на опресняване вероятно е твърде дълъг или кодът блокира между циклите.
- Симптом: разчитането на клавишите е грешно. Проверка: прегледайте съответствието между редове, колони и реалното окабеляване.

Нататък

След това мини към следващата глава, „Минимална FSM и таблична FSM“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част VIII: FSM като начин на мислене

Глава 28. Минимална FSM и таблична FSM

Минимална FSM:

```
STATE_OFF = "OFF" # Това е първото състояние на лампата.
STATE_ON = "ON" # Това е второто състояние на лампата.
EVENT_BUTTON = "BUTTON" # Това е събитието от бутона.

state = STATE_OFF # Започваме с изключена лампа.

def handle_event(current_state, event_name): # Дефинираме преходната функция на машина
та на състояния.
    if current_state == STATE_OFF and event_name == EVENT_BUTTON: # Ако сме в OFF и бу
тонът е натиснат, минаваме в ON.
        return STATE_ON # Връщаме новото състояние ON.
    if current_state == STATE_ON and event_name == EVENT_BUTTON: # Ако сме в ON и буто
нът е натиснат, минаваме в OFF.
        return STATE_OFF # Връщаме новото състояние OFF.
    return current_state # Ако няма преход, оставаме в същото състояние.
```

Таблична FSM:

```
state = "OFF" # Започваме в състояние OFF.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "BUTTON"): "OFF"} # В таблицата оп
исваме преходите като двойка от state и event.

event_sequence = ["BUTTON", "BUTTON", "BUTTON", "BUTTON"] # Симулираме поредица от съб
ития.
```

Графично идеята може да се мисли така:

```
OFF --BUTTON--> ON
ON  --BUTTON--> OFF
```

Файлове:

- examples/30_fsm/01_fsm_lamp_button.py
- examples/30_fsm/02_fsm_table.py

Резюме

- Главата развива системното мислене чрез темата „Минимална FSM и таблична FSM“.
- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „Минимална FSM и таблична FSM“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, guard или правило според темата на главата.
2. Отпечатвайте текущото състояние и последното обработено събитие за по-лесно проследяване.
3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран event/condition.
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помощни функции или таблици.

Нататък

След това мини към следващата глава, „FSM с timeout“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 29. FSM с timeout

Пример:

```
state = "OFF" # Започваме с изключена лампа.

transition_table = {("OFF", "BUTTON"): "ON", ("ON", "TIMEOUT"): "OFF", ("ON", "BUTTON")
: "OFF"} # Описваме преходите с timeout.

event_sequence = ["BUTTON", "TIMEOUT", "BUTTON", "BUTTON"] # Симулираме бутон и timeout събития.
```

Тази глава е преходът от „лампа + бутон“ към „лампа + бутон + време“.

Файл:

- examples/30_fsm/03_fsm_timeout.py

Резюме

- Главата развива системното мислене чрез темата „FSM с timeout“.
- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.

- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „FSM с timeout“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, guard или правило според темата на главата.
2. Отпечатайте текущото състояние и последното обработено събитие за по-лесно проследяване.
3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран event/condition.
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помощни функции или таблици.

Нататък

След това мини към следващата глава, „Conditions vs Events“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 30. Conditions vs Events

Пример:

```
light_level = 600 # Това е моментното ниво от светлинния сензор в симулацията.
dark_threshold = 800 # Това е прагът, под който приемаме че е тъмно.
event_name = "BUTTON" # Това е дискретното събитие от бутон.

condition_is_dark = light_level < dark_threshold # Това е condition, която може да е в
ярна дълго време.

if condition_is_dark and event_name == "BUTTON": # Комбиниране условие и събитие в общ
о решение.
    print("Лампата може да се включи, защото е тъмно и има бутонно събитие.") # Печата
ме положителния изход.
else: # Ако комбинацията не е изпълнена, не включваме лампата.
    print("Няма достатъчно условия за включване.") # Печатаме отрицателния изход.
```

Тук разграничението е:

- condition може да е вярно дълго време
- event е дискретен факт

Файл:

- `examples/30_fsm/04_conditions_vs_events.py`

Резюме

- Главата развива системното мислене чрез темата „Conditions vs Events“.
- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „Conditions vs Events“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от `if` проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, `guard` или правило според темата на главата.
2. Отпечатвайте текущото състояние и последното обработено събитие за по-лесно проследяване.
3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран `event/condition`.
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на `states`, `events` и отделни помощни функции или таблици.

Нататък

След това мини към следващата глава, „Guard функции“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 31. Guard функции

Пример:

```
context = {"light": 500, "pir": True} # Подготвяме контекст със стойности от сензори.
state = "OFF" # Започваме в състояние OFF.
event_name = "MOTION" # Симулираме събитие за засечено движение.

def is_dark(data): # Това е първата guard функция.
    return data["light"] < 800 # Връщаме True само ако е достатъчно тъмно.

def has_motion(data): # Това е втората guard функция.
    return data["pir"] is True # Връщаме True само ако PIR сензорът вижда движение.
```

```
if state == "OFF" and event_name == "MOTION" and is_dark(context) and has_motion(context): # Комбинираме event, state и guard функции.  
    state = "ON" # Минаваме в ON само ако всички проверки са успешни.
```

Тук преходът не зависи само от state и event, а и от допълнителни условия.

Файл:

- `examples/30_fsm/05_guard_functions.py`

Резюме

- Главата развива системното мислене чрез темата „Guard функции“.
- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „Guard функции“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, guard или правило според темата на главата.
2. Отпечатайте текущото състояние и последното обработено събитие за по-лесно проследяване.
3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран event/condition.
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помощни функции или таблици.

Нататък

След това мини към следващата глава, „Приоритети и event queue“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 32. Приоритети и event queue

Приоритети:

```
pending_events = ["BUTTON", "ERROR", "TIMEOUT"] # Симулираме едновременно чакащи събития.  
  
priority_order = {"ERROR": 0, "BUTTON": 1, "TIMEOUT": 2} # По-малкото число означава по-висок приоритет.
```

```
pending_events.sort(key=lambda event_name: priority_order[event_name]) # Подреждаме събитията по приоритет.
```

Event queue:

```
event_queue = [] # Създаваме проста event queue като обикновен Python list.

event_queue.append(("BUTTON", {"source": "SW1"})) # Добавяме събитие от бутон.
event_queue.append(("PIR", {"source": "pir_sensor"})) # Добавяме събитие от PIR сензор
.
event_queue.append(("TIMEOUT", {"ms": 30000})) # Добавяме timeout събитие.

while event_queue: # Обработваме опашката, докато има чакащи събития.
    event_name, payload = event_queue.pop(0) # Взимаме най-старото събитие от началото на опашката.

    print("Обработваме", event_name, "с payload", payload) # Печатаме кое събитие се обработва сега.
```

Това е мостът между:

- прекъсвания
- таймери
- сензори
- FSM логика

Файлове:

- examples/30_fsm/06_priorities.py
- examples/30_fsm/07_event_queue.py

Резюме

- Главата развива системното мислене чрез темата „Приоритети и event queue“.
- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „Приоритети и event queue“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, guard или правило според темата на главата.
2. Отпечатвайте текущото състояние и последното обработено събитие за по-лесно проследяване.

3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран event/condition.
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помощни функции или таблици.

Нататък

След това мини към следващата глава, „Tickless timers и sleep идея“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 33. Tickless timers и sleep идея

Пример:

```
import machine # Импортираме machine за кратък lightsleep между събитията.
import time    # Импортираме time за работа с ticks_ms и ticks_diff.

tasks = [("lamp_timeout", 120), ("fan_timeout", 450), ("display_refresh", 40)] # Подготвяме три задачи с различни крайни срокове в милисекунди.

start_ms = time.ticks_ms() # Запомняме началния момент.

next_deadline_ms = min(deadline for _, deadline in tasks) # Избираме най-близкия срок като tickless цел.

sleep_ms = max(0, next_deadline_ms - time.ticks_diff(time.ticks_ms(), start_ms)) # Изчисляваме колко време може да се спи до следващото събитие.

machine.lightsleep(sleep_ms) # Спим точно до най-близкото планирано събитие с lightsleep.

elapsed_ms = time.ticks_diff(time.ticks_ms(), start_ms) # Изчисляваме изминалото време след съня.

ready_tasks = [task_name for task_name, deadline in tasks if deadline <= elapsed_ms] # Избираме всички задачи, които вече са изтекли.
```

Тази глава не разчита на агресивни sleep режими, а показва начина на мислене:

- намираме най-близкия срок
- спим точно дотогава
- обработваме всички изтекли задачи

Файлове:

- examples/30_fsm/08_tickless_sleep.py
- examples/28_machine_misc/03_sleep_modes.py

Резюме

- Главата развива системното мислене чрез темата „Tickless timers и sleep идея“.

- Основният акцент е върху разделяне на състояния, събития, условия, приоритети и срокове.
- Това е архитектурен материал: целта е не само да работи, а да остане четим и разширяем.

Самопроверка

1. Коя е ключовата моделна идея в „Tickless timers и sleep идея“?
2. Как тази глава прави поведението по-структурирано от чиста поредица от if проверки?
3. Кое решение тук помага най-много за бъдещо разширяване на системата?

Упражнения

1. Добавете още едно състояние, събитие, guard или правило според темата на главата.
2. Отпечатвайте текущото състояние и последното обработено събитие за по-лесно проследяване.
3. Опишете с кратка диаграма или таблица как бихте разширили модела с още един реален вход.

Ако не работи

- Симптом: системата влиза в неочаквано състояние. Проверка: търсете липсващ преход, грешен приоритет или слабо дефиниран event/condition.
- Симптом: моделът става нечетим. Проверка: върнете се към ясни имена на states, events и отделни помощни функции или таблици.

Нататък

След това мини към следващата глава, „Управление на осветление и вентилация за баня“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част IX: Реален пример

Глава 34. Управление на осветление и вентилация за баня

Пример:

```
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

led = Pin("LED1", Pin.OUT, value=1) # LED1 = P204, active-low: value=1 означава изгасен.

button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, active-low: value=0 означава натиснат.

sensors = {"pir": True, "door_open": False, "light_level": 300, "humidity": 82} # Примерни входни данни от четири сензора.

outputs = {"lamp": False, "fan": False} # Два управлявани изхода за лампа и вентилатор.

button_pressed = (button.value() == 0) # SW1 е active-low: 0 означава натиснат.

if sensors["pir"] and sensors["light_level"] < 800: # Ако има движение и е тъмно, включваме лампата.
```

```
outputs["lamp"] = True # Включваме лампата.  
if button_pressed: # Ако бутонът е натиснат, също включваме лампата.  
    outputs["lamp"] = True # Ръчно включване от потребителя.  
if sensors["humidity"] > 75 or sensors["door_open"]: # Ако влажността е висока или вратата е отворена, включваме вентилатора.  
    outputs["fan"] = True # Включваме вентилатора.
```

Това е умишлено учебна симулация, не готов продукт.

Но тя събира:

- движение
- врата
- осветеност
- влажност
- два управлявани изхода

Файл:

- `examples/31_case_study/01_bathroom_control.py`

Резюме

- Главата прилага вече натрупаните идеи в по-реалистичен сценарий чрез „Управление на осветление и вентилация за баня“.
- Тук вече се мисли за цялостна система, а не за изолиран синтактичен или периферен пример.
- Практическата стойност е в това да видите как отделни техники започват да работят заедно.

Самопроверка

1. Коя част от „Управление на осветление и вентилация за баня“ е реална и коя е умишлено симулирана за учебни цели?
2. Кои решения в тази глава са добри за учебен пример, но още не са завършен продукт?
3. Кой предишен урок се използва най-видимо тук?

Упражнения

1. Променете един праг или правило и опишете как това влияе на цялата система.
2. Добавете кратък лог на решенията, които системата взема.
3. Разделете една част от логиката в помощна функция, за да направите кода по-четим.

Ако не работи

- Симптом: логиката изглежда твърде сложна. Проверка: проследявайте отделно входовете, условията и изходите, вместо да гледате всичко наведнъж.
- Симптом: резултатът не отговаря на очакванията. Проверка: вижте кои входове са реални и кои са само симулирани данни в примера.

Нататък

След това мини към следващата глава, „Декомпозиция на две FSM“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 35. Декомпозиция на две FSM

Пример:

```
lamp_state = "OFF" # Това е текущото състояние на FSM за лампата.
fan_state = "OFF" # Това е текущото състояние на FSM за вентилатора.

event_sequence = [] # Започваме с празен списък.

if button.value() == 0: # Проверяваме дали SW1 е натиснат в момента.
    event_sequence.append("BUTTON") # Добавяме реално събитие от бутона.

event_sequence.extend(["HUMIDITY_HIGH", "TIMEOUT_LAMP", "TIMEOUT_FAN"]) # Добавяме симулирани събития.

for event_name in event_sequence: # Обработваме събитията последователно и за двете машини.
    if event_name == "BUTTON": # Бутонът управлява лампата.
        lamp_state = "ON" if lamp_state == "OFF" else "OFF" # Превключваме FSM на лампата.
    if event_name == "HUMIDITY_HIGH": # Високата влажност влияе на FSM на вентилатора.
        fan_state = "ON" # Включваме вентилатора.
```

Тук голямата идея е:

- една FSM да не поглъща цялата система
- отделните машини да имат отделни отговорности
- комуникацията да става през събития

Файл:

- `examples/31_case_study/02_two_fsms.py`

Забележка: И двата case study примера вече използват реален LED1 (P204) и SW1 (P400) за вход и изход, освен симулираните сензори.

Резюме

- Главата прилага вече натрупаните идеи в по-реалистичен сценарий чрез „Декомпозиция на две FSM“.
- Тук вече се мисли за цялостна система, а не за изолиран синтактичен или периферен пример.
- Практическата стойност е в това да видите как отделни техники започват да работят заедно.

Самопроверка

1. Коя част от „Декомпозиция на две FSM“ е реална и коя е умишлено симулирана за учебни цели?

2. Кой решения в тази глава са добри за учебен пример, но още не са завършен продукт?
3. Кой предишен урок се използва най-видимо тук?

Упражнения

1. Променете един праг или правило и опишете как това влияе на цялата система.
2. Добавете кратък лог на решенията, които системата взема.
3. Разделете една част от логиката в помощна функция, за да направите кода по-четим.

Ако не работи

- Симптом: логиката изглежда твърде сложна. Проверка: проследявайте отделно входовете, условията и изходите, вместо да гледате всичко наведнъж.
- Симптом: резултатът не отговаря на очакванията. Проверка: вижте кои входове са реални и кои са само симулирани данни в примера.

Нататък

След това мини към следващата глава, „Базова UART комуникация“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Част X: UART комуникация

Глава 36. Базова UART комуникация

Пример:

```
from machine import UART # Импортираме UART класа за серийна комуникация.
import time # Импортираме time за кратко закъснение.

uart = UART(9, 115200) # Създаваме UART(9) на 115200 baud (TX=P602, RX=P601).
message = "Здравей от VK_RA4M2!\r\n" # Подготвяме съобщение за изпращане.
bytes_sent = uart.write(message) # Изпращаме съобщението по TX пина.

time.sleep_ms(100) # Кратко закъснение, за да пристигнат данни (ако има loopback).
available = uart.any() # Проверяваме дали има налични байтове за четене.
if available > 0: # Ако има данни за четене:
    received = uart.read(available) # Четем наличните байтове от RX буфера.
    print("Получено:", received) # Показваме получените данни.
```

Налични UART канали:

- UART(0) -> TX=P411, RX=P410
- UART(2) -> TX=P302, RX=P301
- UART(7) -> TX=P401, RX=P402, CTS=P403
- UART(9) -> TX=P602, RX=P601, CTS=P603

Тук се учат:

- създаване на UART инстанция
- изпращане и четене на данни
- loopback тест (TX свързан към RX)

Файл:

- `examples/32_uart/01_uart_basic.py`

Резюме

- Главата въвежда UART като най-простия сериен канал за текст и байтове.
- Показани са базово изпращане, наличие на данни и четене от буфера.
- Loopback тестът е естественият първи sanity check преди по-сложна комуникация.

Самопроверка

1. Каква е ролята на baud rate при UART комуникацията?
2. Защо loopback тестът е добър първи тест на конфигурацията?
3. Каква е практическата разлика между `any()` и `read()`?

Упражнения

1. Изпращайте прост текстов брояч веднъж в секунда по UART.
2. Ако имате loopback, отпечатвайте обратно полученото съобщение като echo тест.
3. Добавете функция за изпращане на текстов ред с край `\r\n`.

Ако не работи

- Симптом: текстът е нечетим. Проверка: baud rate и настройките от двете страни трябва да съвпадат.
- Симптом: няма върнати данни. Проверка: при loopback TX и RX трябва да са свързани по очаквания за теста начин.

Нататък

След това мини към следващата глава, „Работа с външни модули по интерфейси“, за да надградиш текущата идея от голо серийно предаване към реална интеграция на модул.

Част XI: Външни модули по интерфейси

Глава 37. Работа с външни модули по интерфейси

Тази глава не въвежда нов хардуерен контролер.

Тя събира вече видените I2C, SPI и UART в един практичен модел за мислене.

Когато включвате външен модул, не започвате от „готов driver“, а от четири по-базови въпроса:

1. С какво напрежение и логически нива работи модулът?

2. Кой интерфейс използва: I2C, SPI или UART?
3. Има ли допълнителни control линии като CS, RST, INT, BUSY или EN?
4. Кой е най-малкият безопасен probe, който доказва, че връзката работи?

На VK_RA4M2 има три особено важни практични ограничения:

- I2C(0) дели P400 със SW1
- I2C(1) дели P100/P101 със hardware SPI
- UART(9) е най-удобният отделен канал, когато I2C или SPI вече са заети

Това значи, че изборът на интерфейс не е само логически, а и ресурсен.

Кога кой интерфейс е най-подходящ:

- I2C е удобен за сензори и по-бавни модули с адресирана обща шина
- SPI е удобен за по-бързи модули, при които имате отделен CS и frame-базиран обмен
- UART е удобен за модеми, GPS, Bluetooth и други модули с текстов или байтов поток

Практическият ред на работа почти винаги е един и същ:

1. Свържете GND и проверете напрежението
2. Проверете дали логическите нива на модула са съвместими с борда
3. Свържете сигналните линии на интерфейса
4. Свържете и нужните control линии
5. Стартирайте с минимална скорост и минимален probe
6. Едва след това четете идентификатор, статус или payload

Минимален I2C probe:

```
from machine import I2C # Импортираме I2C за базов probe на външна шина.  
  
i2c = I2C(1, freq=100000) # Създаваме I2C(1) на безопасни 100 kHz.  
print(i2c.scan()) # Сканираме шината за адреси на свързани I2C модули.
```

Минимална SPI транзакция:

```
from machine import SPI, Pin # Импортираме SPI и Pin за frame-базиран обмен с външен м  
одул.  
  
cs = Pin("P103", Pin.OUT, value=1) # Държим CS неактивен преди началото на транзакция  
та.  
  
spi = SPI(0, baudrate=500000, polarity=0, phase=0, bits=8, sck=Pin("P102"), mosi=Pin("P  
101"), miso=Pin("P100"), cs=cs) # Създаваме SPI с консервативни настройки.  
  
cs.value(0) # Активираме target модула чрез CS.  
  
spi.write_readinto(b"\x9F\x00\x00\x00", bytearray(4)) # Изпращаме команда и четем едно  
временно отговора.
```

```
cs.value(1) # Освобождаваме модула след края на frame-a.
```

Минимален UART request/response:

```
from machine import UART # Импортираме UART за сериен обмен с външен модул.  
uart = UART(9, 115200) # Създаваме UART(9) с често срещана начална скорост.  
uart.write(b"AT\r\n") # Изпращаме примерна текстова команда към модула.  
print(uart.read()) # Четем каквото е пристигнало към този момент.
```

Общата идея е проста:

- I2C първо се доказва със `scan()`
- SPI първо се доказва с правилен CS и пълен frame
- UART първо се доказва с правилен baud rate, TX/RX и timeout

След това вече се минава към конкретния регистров модел или към конкретния текстов протокол на модула.

Тук много по-важен от „готовия код“ е datasheet-ът:

- за I2C ви трябват адрес и регистри
- за SPI ви трябват polarity, phase, CS логика и формат на frame-a
- за UART ви трябват baud rate, формат на кадъра и синтаксис на командите

Файлове:

- `examples/34_external_modules/01_i2c_module_probe.py`
- `examples/34_external_modules/02_spi_module_transaction.py`
- `examples/34_external_modules/03_uart_module_request_response.py`

Резюме

- Главата събира I2C, SPI и UART в единен practically-first подход за интеграция на външни модули.
- Фокусът е върху захранване, логически нива, control линии и най-малкия възможен probe.
- На VK_RA4M2 изборът на интерфейс е свързан и с конфликти между пинове и периферии.

Самопроверка

1. Защо при външен модул е грешно да започнете директно от голям driver, без минимален probe?
2. Кога бихте избрали SPI вместо I2C за външен модул?
3. Защо UART(9) често е най-удобният избор, когато вече ползвате I2C(1) или SPI?

Упражнения

1. Направете собствен checklist за свързване на нов външен модул с колони power, pins, protocol, probe.

2. Разширете `01_i2c_module_probe.py`, така че да четете два различни регистра, ако адресът е намерен.
3. Добавете `timeout` и втори опит в `03_uart_module_request_response.py`, преди да обявите връзката за неуспешна.

Ако не работи

- Симптом: модулът мълчи напълно. Проверка: първо вижте GND, захранването и логическите нива, не кода.
- Симптом: има отговор, но той е безсмислен. Проверка: вижте адреса, `baud rate`, `SPI mode`, формата на командата и `datasheet-a`.
- Симптом: модулът тръгва само понякога. Проверка: проверете RST, CS, BUSY, INT и ресурсните конфликти между пиновете на борда.

Нататък

След това мини към следващата глава, „Основи на `asyncio`“, за да видиш как външен модул може да се обслужва неблокиращо и като част от по-голямо приложение.

Част XII: `asyncio` като учебна секция

Глава 38. Основи на `asyncio`

Пример:

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.

async def blink_task(period_ms): # Асинхронна задача за мигане с зададен период.
    while True: # Безкрайно мигане.
        led.value(LED_ON) # Светваме LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението за period_ms.
        led.value(LED_OFF) # Гасим LED1.
        await asyncio.sleep_ms(period_ms) # Отстъпваме управлението отново.
```

Тази глава покрива:

- `create_task` за стартиране на задачи
- `asyncio.gather` за паралелно изпълнение
- три задачи едновременно: мигане, бутон, отчет

Файл:

- `examples/33_asyncio/01_asyncio_basics.py`

Резюме

- Главата систематизира `asyncio` чрез няколко кооперативни задачи.
- Показва се как LED, бутон и отчет могат да живеят като отделни роли в една програма.
- Това е реален практически модел за неблокиращи приложения върху платката.

Самопроверка

1. Какво прави `create_task` спрямо директно извикване на функция?
2. Защо `asyncio` е кооперативен, а не прекъсващ модел?
3. Кога бихте избрали този подход пред много блокиращи цикли?

Упражнения

1. Добавете нова `async` задача, която периодично логва състоянието на приложението.
2. Променете периода на мигане според някакво събитие или брояч.
3. Спрете една от задачите след `timeout` и проверете дали останалите продължават.

Ако не работи

- Симптом: `event loop`-ът „замръзва“. Проверка: някоя задача вероятно съдържа блокиращ код без `await`.
- Симптом: дадена задача сякаш не съществува. Проверка: вижте дали реално е създадена и дали отстъпва управлението.

Нататък

След това мини към следващата глава, „`asyncio` Event синхронизация“, за да надградиш текущата идея в естествената ѝ следваща стъпка.

Глава 39. `asyncio` Event синхронизация

Пример:

```
import asyncio # Импортираме asyncio за кооперативна многозадачност.
from machine import Pin # Импортираме Pin за реален хардуерен достъп.

LED_ON = 0 # Active-low: 0 = светва LED1.
LED_OFF = 1 # Active-low: 1 = гаси LED1.

led = Pin("LED1", Pin.OUT, value=LED_OFF) # LED1 = P204, започваме изгасен.
button = Pin("SW1", Pin.IN, Pin.PULL_UP) # SW1 = P400, вътрешен pull-up.
button_event = asyncio.Event() # Създаваме Event обект за синхронизация между задачи.

async def button_producer(): # Тази задача следи SW1 и сигнализира при натискане.
    last_value = 1 # Последна стойност на бутона (ненатиснат).
```

```
current_value = button.value() # Четем SW1.  
if current_value == 0 and last_value == 1: # Нов натиск (falling edge).  
    button_event.set() # Сигнализираме Event-а.  
  
async def led_consumer(): # Тази задача чака Event и мига LED1.  
    await button_event.wait() # Чакаме Event-а (блокираме кооперативно).  
    button_event.clear() # Нулираме Event-а за следващо използване.
```

Тук се вижда:

- `asyncio.Event` за комуникация между задачи
- производител-консуматор модел
- кооперативно блокиране без `busy-wait`

Файл:

- `examples/33_asyncio/02_asyncio_event_flag.py`

Резюме

- Главата показва `asyncio.Event` като чист механизъм за синхронизация между задачи.
- Моделът `producer-consumer` прави асинхронната комуникация по-ясна и по-надеждна.
- Това е важна крачка от „много задачи“ към „задачи, които си взаимодействат правилно“.

Самопроверка

1. Каква е ползата от `asyncio.Event` спрямо обикновен споделен флаг?
2. Защо консуматорът обикновено изчиства Event-а след обработка?
3. Как бихте разширили примера, ако няколко задачи трябва да реагират на едно събитие?

Упражнения

1. Добавете брояч на Event сработванията и го отпечатвайте периодично.
2. Създайте втори консуматор, който също чака същото събитие.
3. Добавете `debounce` или `edge detection` в `producer` задачата преди `set()`.

Ако не работи

- Симптом: реакцията се случва само веднъж. Проверка: вижте дали Event-ът отново се задава след `clear()`.
- Симптом: натискания се губят. Проверка: `producer` логиката и `polling` интервалът може да са твърде груби.

Нататък

След тази глава прегледайте каталога на примерите и се върнете към темите, които искате да доразвиете на реален борд.

Част XIII: Специализирани светлинни модули

Глава 40. NeoPixel

NeoPixel е удобен пример за „умен“ външен RGB светодиоден модул.

Той не се управлява като обикновен RGB LED с три PWM канала.

Вместо това се използва един data проводник с точно времедиаграмиране, което MicroPython крие зад модула neopixel.

В този курс използваме:

- P500 като малко захранване за единичен модул
- P112 като DIN data линия
- 1 пиксел за най-простия тест

Това е удобно за демонстрация, но има важно ограничение:

- за единичен модул или много кратка лента може да е достатъчно
- за по-дълга лента трябва отделно външно захранване и обща маса

Ключовата идея е, че работата с NeoPixel минава през три стъпки:

1. Включвате захранването на модула
2. Записвате RGB стойност в буфера
3. Извиквате `write()`, за да се изпрати `timed bitstream` към пиксела

Минимален пример:

```
from machine import Pin # Импортираме Pin за power и data линията.
import neopixel # Импортираме neopixel драйвера за RGB модула.

vcc = Pin("P500", Pin.OUT, value=1) # Подаваме захранване към малкия външен модул.
np = neopixel.NeoPixel(Pin("P112"), 1) # Създаваме NeoPixel обект за един пиксел върху P112.

np[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.
np.write() # Изпращаме буфера и реално сменяме цвета на модула.
```

Тук има две практични разлики спрямо обикновен GPIO или PWM пример:

- цветът се задава като кортеж (R, G, B)
- промяната не се вижда преди `write()`

Втори важен практически момент е яркостта.

В учебните примери стойностите са ниски, например 40, за да се ограничи токът и да се намали рискът при захранване на единичен модул от GPIO.

Трети важен момент е, че NeoPixel е външен модул, така че физическото свързване е толкова важно, колкото и кодът:

- DIN на модула трябва да отиде към P112
- захранването трябва да е стабилно
- ако се ползва външно захранване, масата трябва да е обща с борда

Файлове:

- `examples/neopixel.py`
- `examples/neopixel_test.py`

Резюме

- Главата показва NeoPixel като специализиран външен RGB модул с еднопроводен data интерфейс.
- Основният practically-first модел е power, запис в буфера и `write()`.
- Ниската яркост и правилното захранване са критични за безопасен първи тест.

Самопроверка

1. Защо NeoPixel не се управлява като обикновен RGB LED с три отделни PWM канала?
2. Каква е ролята на `write()` след присвояване на `np[0] = (...)`?
3. Защо при по-дълга LED лента не е добра идея да разчитате на P500 като захранване?

Упражнения

1. Увеличете броя пиксели на 2 или 4 и направете цветово преливане по лентата.
2. Добавете функция, която изгасва всички пиксели след определено време без действие.
3. Свържете SW1 и сменяйте цвета на NeoPixel-а при всяко натискане.

Ако не работи

- Симптом: няма никаква светлина. Проверка: вижте дали P500 реално подава захранване и дали DIN е към правилния вход на модула.
- Симптом: цветът е грешен или нестабилен. Проверка: намалете яркостта, скъсете проводниците и проверете общата маса.
- Симптом: единичният модул работи, а лентата не. Проверка: при повече пиксели минете към отделно външно захранване.

Нататък

След тази глава прочетете и следващата глава за WS2812 през `machine.WS2812`, за да видите разликата между софтуерен bitstream и SCI-базиран TX-only подход.

Глава 41. WS2812 през `machine.WS2812` и SCI TX-only

WS2812B е от същото семейство адресируеми RGB светодиоди, но в този порт вече има и втори начин за управление.

Вместо класическия neopixel модул с `machine.bitstream()`, може да се използва `machine.WS2812`.

Този драйвер кодира данните през подходящ SCI канал и извежда само data сигнала през избрания TXD/MOSI pin.

Практически това означава:

- към самия светодиоден модул свързвате само DIN към избрания data pin
- не подавате външен SCK и MISO към лентата
- продължавате да работите с буфер и `write()`, почти както при NeoPixel

От гледна точка на Python кодът е еднопроводен: има само data pin към модула.

От гледна точка на микроконтролера вътрешно се използва SCI хардуерът, така че избраният pin автоматично определя кой SCI блок ще бъде зает.

Поддържаните data pin-ове в този порт са:

- SCI0 -> P101, P205, P411
- SCI1 -> P213, P401, P501
- SCI2 -> P102, P112, P302
- SCI9 -> P109, P203, P409, P602

Това значи, че P112 вече е default pin, но не е единствен възможен.

Важното ограничение е по канали, не само по pin.

Ако изберете pin от SCI2 групата, не трябва едновременно да очаквате:

- `machine.WS2812`
- UART(2)
- SPI(2) през SCI2

Същата логика важи и за другите групи, ако вече ползвате същия SCI канал за друг интерфейс.

Минимален пример с алтернативен pin:

```
from machine import Pin # Импортираме Pin за малкото захранване и за data линията.
from machine import WS2812 # Импортираме специализирания драйвер за WS2812 с runtime p
in избор.

vcc = Pin("P500", Pin.OUT, value=1) # Подаваме малко захранване към единичния външен м
одул.

strip = WS2812(pixel_count=1, pin=Pin("P109"), channels=3) # Създаваме драйверен обект
върху алтернативния pin P109.

strip[0] = (40, 0, 0) # Зареждаме червено в буфера на първия пиксел.

strip.write() # Изпраща буфера към модула през SCI TX-only backend-a.
```

Подобно на `NeoPixel`, и тук цветът не се вижда преди `write()`.

Разликата е, че тук `timing`-ът не идва от общия `bitstream` механизъм, а от специализиран C-side драйвер в порта.

Това е полезно по три причини:

- API-то е ясно вързано към `WS2812`
- можете да изберете удобен `SCI TX/MOSI pin` според реалното окабеляване
- логиката за кодиране и `latch` остава в C, а не в Python

Важното ограничение е, че този драйвер е порт-специфичен.

Тоест примерите са силно полезни точно за `VK_RA4M2`, но не бива да очаквате същия `machine.WS2812` API на произволен друг `MicroPython` порт.

Файлове:

- `examples/ws2812_sci.py`
- `examples/ws2812_sci_test.py`
- `examples/ws2812_sci_p101.py`
- `examples/ws2812_sci_p109.py`

Резюме

- Главата показва `WS2812` като специализиран `machine` клас, реализиран в този порт върху няколко `SCI TX/MOSI pin` групи.
- За самото свързване към модула е нужен само избраният `data pin`, без външен `clock pin` към светодиода.
- Конфликтите са по `SCI` канал: не може един и същ `SCI` блок да се дели едновременно от `WS2812` и `UART` или `SPI`.

Самопроверка

1. Каква е разликата между `neopixel.NeoPixel` и `machine.WS2812` в този порт?
2. Защо потребителят свързва само `DIN` -> избрания `TXD/MOSI pin`, въпреки че вътрешно драйверът използва `SCI`?
3. Какво общо имат `P109` и `UART(9)` при използване на `machine.WS2812`?

Упражнения

1. Пуснете един и същ пример веднъж с `P112`, а после с `P109`, и сравнете нужните промени в Python кода.
2. Променете `pixel_count` на 2 или 4 и направете цветен „марш“ по къса лента.
3. Изберете `pin` от `SCI0` или `SCI1` групата и проверете кои други интерфейси вече не бива да активирате едновременно.

Ако не работи

- Симптом: няма никаква светлина. Проверка: вижте дали P500 реално подава захранване и дали DIN е към правилния вход на модула.
- Симптом: получавате грешка за зает bus. Проверка: избраният SCI канал може вече да е зает от UART или SPI със същия номер.
- Симптом: цветовете са нестабилни при повече пиксели. Проверка: минете към отделно външно захранване и обща маса.
- Симптом: neopixel примерът работи, а този не. Проверка: това е индикация, че точно избраният SCI TX/MOSI path трябва да се валидира на реален хардуер.

Нататък

След тази глава прегледайте каталога на примерите и изберете дали да останете с универсалния neopixel подход, или с порт-специфичния machine.WS2812 с runtime избор на data pin.

Каталог на всички примери

Този каталог е финалната проверка, че нито един пример не е пропуснат в книгата.

- `examples/01_blink_async.py` : асинхронно мигане на LED1
- `examples/02_two_tasks.py` : две едновременни `asyncio` задачи
- `examples/03_pwm_ab_same_channel.py` : два PWM изхода върху един GPT канал
- `examples/04_touchpad_async.py` : асинхронно следене на TouchPad с `cached API`
- `examples/neopixel.py` : NeoPixel rainbow cycle с P500 захранване и P112 data линия
- `examples/neopixel_test.py` : кратък NeoPixel smoke test с основни цветове и изгасване
- `examples/ws2812_sci.py` : WS2812 rainbow cycle през machine.WS2812 и SCI2 върху P112
- `examples/ws2812_sci_test.py` : кратък WS2812 smoke test през machine.WS2812
- `examples/ws2812_sci_p101.py` : WS2812 rainbow cycle през machine.WS2812 върху алтернативния SCI0 pin P101
- `examples/ws2812_sci_p109.py` : кратък WS2812 smoke test през machine.WS2812 върху алтернативния SCI9 pin P109
- `examples/20_language_basics/01_sample_program.py` : първа примерна програма
- `examples/20_language_basics/02_variables_and_constants.py` : променливи и константи
- `examples/20_language_basics/03_operators_conditions_loops.py` : оператори, условия и цикли
- `examples/20_language_basics/04_enums.py` : enum-подобни стойности с `const()`
- `examples/20_language_basics/05_arrays.py` : списъци, вложени таблици, `array`, `bytearray`
- `examples/20_language_basics/06_structures.py` : `dict`, `namedtuple`, вложени структури
- `examples/20_language_basics/07_pointers_equivalents.py` : `references`, `memoryview`, `machine.mem32`
- `examples/20_language_basics/08_functions.py` : функции, `callbacks` и таблици от функции
- `examples/21_digital_io/01_digital_output.py` : цифров изход с LED1

- `examples/21_digital_io/02_digital_input.py` : цифров вход с SW1
- `examples/22_analog/01_analog_input_adc.py` : ADC вътрешни и външни канали
- `examples/22_analog/02_pwm_output.py` : базов PWM изход
- `examples/23_timing/01_delays_and_timers.py` : блокиращо закъснение, хардуерен таймер, `uasyncio`
- `examples/23_timing/02_interrupts.py` : GPIO прекъсвания и защитено четене
- `examples/23_timing/03_button_debounce.py` : дебаунс с `polling` и с `uasyncio`
- `examples/23_timing/04_software_timer_minus1.py` : софтуерен `Timer(-1)`
- `examples/23_timing/05_timer_compare_hw_sw.py` : сравнение между `Timer(1)` и `Timer(-1)`
- `examples/23_timing/06_rtc_basic.py` : RTC основи
- `examples/23_timing/07_adc_timing_measure.py` : измерване на `ADC.read()` с `time.ticks_us()`
- `examples/23_timing/08_interrupt_latency_measure.py` : ориентировъчно измерване на GPIO interrupt latency
- `examples/23_timing/09_button_polling_events.py` : `polling` четене на бутон със `PRESS`, `RELEASE`, `CLICK` и `LONG_PRESS`
- `examples/23_timing/10_button_irq_deferred.py` : бутон по прекъсване с отложена `debounce` обработка
- `examples/23_timing/11_button_soft_timer_hold.py` : `Timer(-1)` sampler с `PRESS`, `RELEASE`, `HOLD_START` и `HOLD_REPEAT`
- `examples/23_timing/12_button_shift_debounce.py` : `debounce` чрез битово изместване и `shift-register` история
- `examples/24_i2c/01_i2c_master_basic.py` : базов I2C master
- `examples/24_i2c/02_i2c_runtime_pins.py` : I2C с runtime зададени пинове
- `examples/24_i2c/03_softi2c_basic.py` : `SoftI2C`
- `examples/24_i2c/04_i2ctarget_memory.py` : `I2CTarget` с `memory-backed` буфер
- `examples/25_spi/01_spi_basic.py` : hardware SPI
- `examples/25_spi/02_softspi_basic.py` : `SoftSPI`
- `examples/26_touchpad/01_touchpad_basic.py` : базов `TouchPad`
- `examples/26_touchpad/02_touchpad_diagnostics.py` : диагностика, `tuning` и `cached API`
- `examples/27_storage/01_flash_blockdev.py` : `renesas.Flash` и `/flash`
- `examples/27_storage/02_dataflash_basic.py` : `dataflash` модул
- `examples/27_storage/03_dataflash_state_led.py` : устойчиво състояние в `dataflash` и LED индикация
- `examples/28_machine_misc/01_machine_info_and_id.py` : `machine.info`, `freq`, `unique_id`
- `examples/28_machine_misc/02_machine_mem_and_bootloader.py` : `disable_irq`, `enable_irq`, `mem32`, `bootloader`
- `examples/28_machine_misc/03_sleep_modes.py` : `sleep`, `lightsleep`, `deepsleep`

- `examples/28_machine_misc/04_memory_management.py` : `gc`, `micropython.mem_info`, `bytearray`, `memoryview`
- `examples/29_display_scan/01_dynamic_7seg_and_keypad.py` : динамична индикация и матрично сканиране
- `examples/30_fsm/01_fsm_lamp_button.py` : минимална FSM
- `examples/30_fsm/02_fsm_table.py` : таблична FSM
- `examples/30_fsm/03_fsm_timeout.py` : FSM с timeout
- `examples/30_fsm/04_conditions_vs_events.py` : conditions срещу events
- `examples/30_fsm/05_guard_functions.py` : guard функции
- `examples/30_fsm/06_priorities.py` : приоритети между събития
- `examples/30_fsm/07_event_queue.py` : event queue
- `examples/30_fsm/08_tickless_sleep.py` : tickless идея и sleep
- `examples/31_case_study/01_bathroom_control.py` : баня с няколко сензора и реален LED1/SW1
- `examples/31_case_study/02_two_fsms.py` : лампа и вентилатор като две FSM с реален LED1/SW1
- `examples/32_uart/01_uart_basic.py` : базова UART комуникация и loopback тест
- `examples/34_external_modules/01_i2c_module_probe.py` : I2C scan и безопасен probe на външен модул
- `examples/34_external_modules/02_spi_module_transaction.py` : SPI framed transaction с CS и анализ на суровия отговор
- `examples/34_external_modules/03_uart_module_request_response.py` : UART request/response с timeout към външен модул
- `examples/33_asyncio/01_asyncio_basics.py` : asyncio основи с три паралелни задачи
- `examples/33_asyncio/02_asyncio_event_flag.py` : asyncio Event синхронизация
- `examples/index.py` : скрипт за листване на всички примери на борда

Какво още не е потвърдено на реален борд

Книгата и примерите са прегледани за пълност и са подредени спрямо плана.

Още не е потвърдено пълно runtime изпълнение на всички примери върху реален VK_RA4M2.

Най-чувствителните места остават:

- външните I2C устройства
- SPI loopback и външни SPI модули
- външните UART модули и AT-подобни текстови протоколи
- NeoPixel модулт върху P500/P112, особено при по-дълги проводници или повече от един пиксел

- WS2812 path-ът през `machine.WS2812` и различните `SCI TX/MOSI` pin-ове, защото е нов порт-специфичен backend и трябва да се потвърди на реален хардуер
- 7-сегментната индикация и матричната клавиатура
- write режимите на Flash и dataflash
- lightsleep и deepsleep

Заклучение

Тази версия на книгата е построена като проверка за пълност:

- всички теми от плана са отразени
- всички 67 примерни файла са споменати изрично
- всяка голяма тема има поне един реален кодов откъс
- порт-специфичните ограничения на VK_RA4M2 са записани ясно

Следващата разумна стъпка вече не е „още съдържание“, а хардуерна валидация на примерите по групи върху реалната платка.